

20 PRODUCTION-GRADE KUBERNETES OPERATOR AND CONTROLLER PROJECTS



REAL-WORLD SCENARIOS



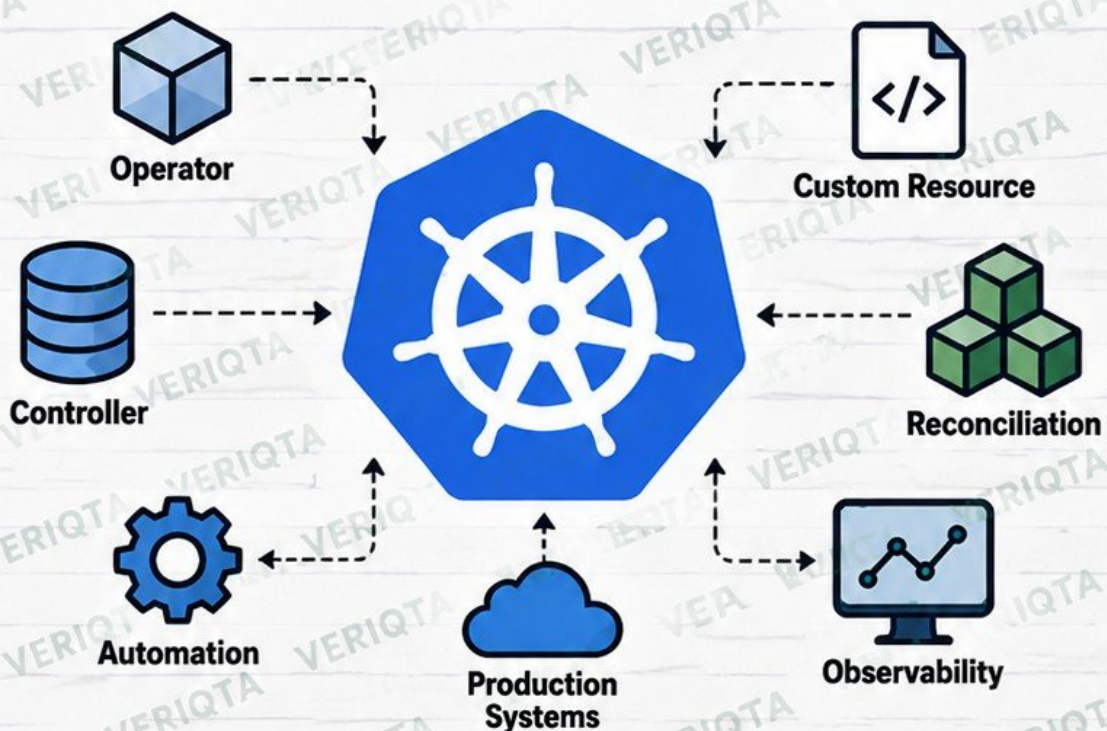
PRODUCTION PATTERNS



TROUBLESHOOTING AND FAILURE TESTING



SCALABLE ARCHITECTURES



BUILD

ADVANCED OPERATORS



AUTOMATE

PRODUCTION WORKFLOWS



SECURE

YOUR PLATFORM



SCALE

REAL-WORLD INFRASTRUCTURE



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

01 Build a Multi-Tenant PostgreSQL Database Operator

Automate provisioning, credential rotation, backups, upgrades, and safe deletion through custom resources.

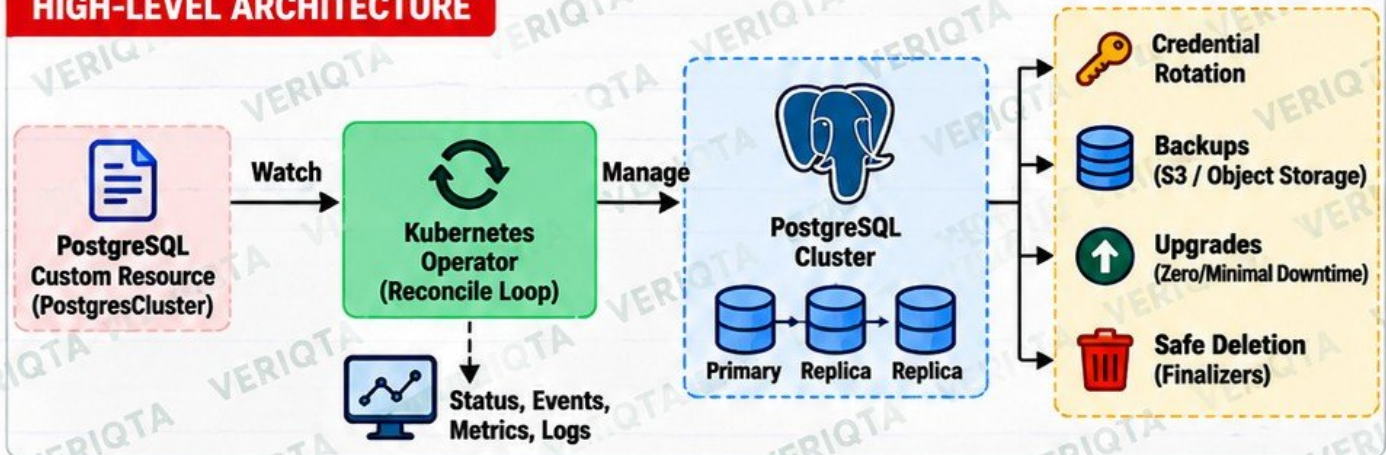
PROJECT OBJECTIVE

- ✓ Provision PostgreSQL for multiple tenants
- ✓ Automate user and credential rotation
- ✓ Manage backups and recovery
- ✓ Handle version upgrades safely
- ✓ Ensure secure and isolated deployments
- ✓ Support safe and clean deletion

KEY TECHNOLOGIES



HIGH-LEVEL ARCHITECTURE



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1      # Custom API group
kind: PostgresCluster         # Resource type
metadata:
  name: tenant-a-db           # Cluster name
  namespace: tenant-a         # Tenant namespace
spec:
  version: "15"               # PostgreSQL version
  storage:
    size: 100Gi               # Persistent storage size
  users:
    - name: appuser           # Application user
      privileges: ["read", "write"]
  backup:
    enabled: true              # Backup configuration
    schedule: "0 2 * * *"     # Daily at 2 AM
    retention: 30d             # Keep 30 days
  upgrade:
    strategy: "rolling"        # Upgrade configuration
    version: "16"
```

RECONCILIATION FLOW

- 1 Read desired state from PostgresCluster CR
- 2 Validate configuration and tenant isolation
- 3 Provision or update PostgreSQL resources
- 4 Manage users, passwords and credential rotation
- 5 Configure and verify backups
- 6 Handle version upgrades safely
- 7 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use finalizers to prevent orphaned resources
- ✓ Ensure tenant isolation with separate namespaces and RBAC
- ✓ Implement secure credential rotation with zero downtime
- ✓ Verify backup integrity and restore regularly
- ✓ Handle failures with retries and idempotent reconciliation
- ✓ Monitor with Prometheus metrics and Kubernetes events
- ✓ Support version upgrades with minimal service interruption



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

02 Build a Zero-Downtime Database Schema Migration Controller

Coordinate schema changes with application releases using expand-contract migrations, compatibility checks, and rollback boundaries.

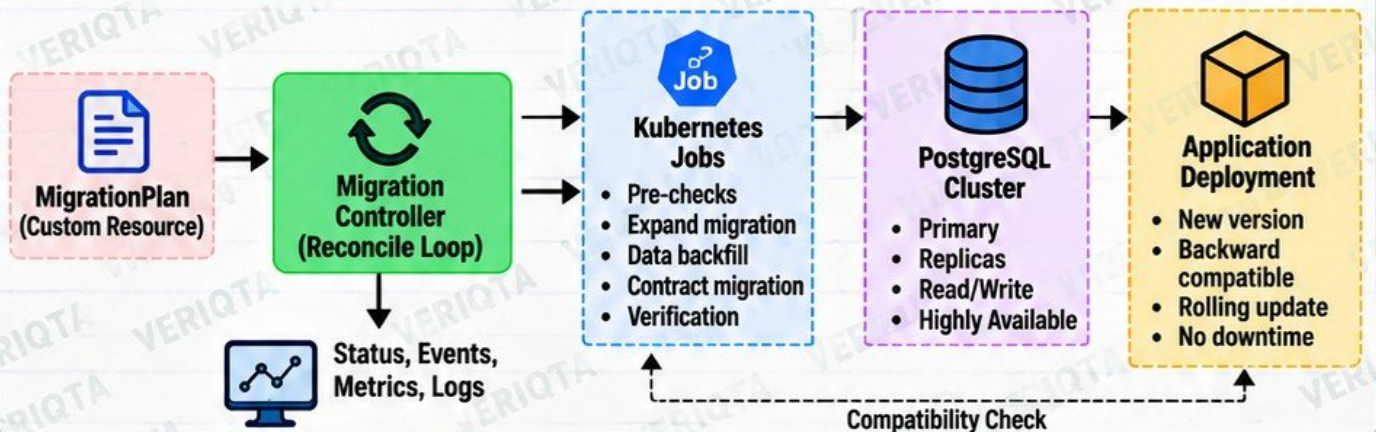
PROJECT OBJECTIVE

- ✓ Automate safe, zero-downtime schema migrations
- ✓ Use expand-contract migration strategy
- ✓ Validate application and database compatibility
- ✓ Coordinate migrations with application releases
- ✓ Provide rollback boundaries and failure handling
- ✓ Observability through status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```

apiVersion: veriqta.io/v1 # API version
kind: MigrationPlan # Resource type
metadata:
  name: add-user-email # Name of migration plan
  namespace: tenant-a # Namespace
spec:
  databaseRef: # Database to migrate
    name: tenant-a-db
  applicationRef: # Application using this DB
    name: user-service
  strategy: expand-contract # Expand → backfill → contract
  migrations:
    expand: # SQL migration files
      - file: 001_add_email_column.sql
    backfill:
      - file: 002_backfill_email.sql
    contract:
      - file: 003_set_not_null.sql
  checks: # Pre-migration checks
    - type: app-compatibility
    - type: data-validation
  rollback: # Rollback configuration
    enabled: true
    strategy: manual
  
```

RECONCILIATION FLOW

- 1 Validate MigrationPlan and tenant permissions
- 2 Check application and database compatibility
- 3 Run pre-migration checks
- 4 Execute expand migration (safe and backward compatible)
- 5 Run data backfill (if required)
- 6 Verify application readiness
- 7 Execute contract migration
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use expand-contract strategy for zero downtime
- ✓ Validate app compatibility before each phase
- ✓ Make migrations idempotent and retry-safe
- ✓ Handle partial failures with clear rollback boundaries
- ✓ Monitor database performance during migrations
- ✓ Support multi-tenant isolation and RBAC
- ✓ Provide approval gates for high-risk changes

FAILURE SCENARIOS

- ! Migration job fails mid-way
- ! Application not compatible
- ! Long running backfill
- ! Lock contention on tables
- ! Insufficient disk space
- ! Replicas falling behind
- ! Controller restart during migration



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

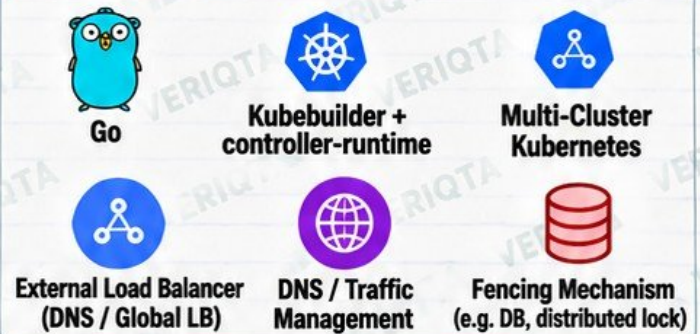
03 Build a Multi-Cluster Application Failover Operator

Detect unhealthy clusters, coordinate traffic failover, enforce fencing, and prevent split-brain recovery.

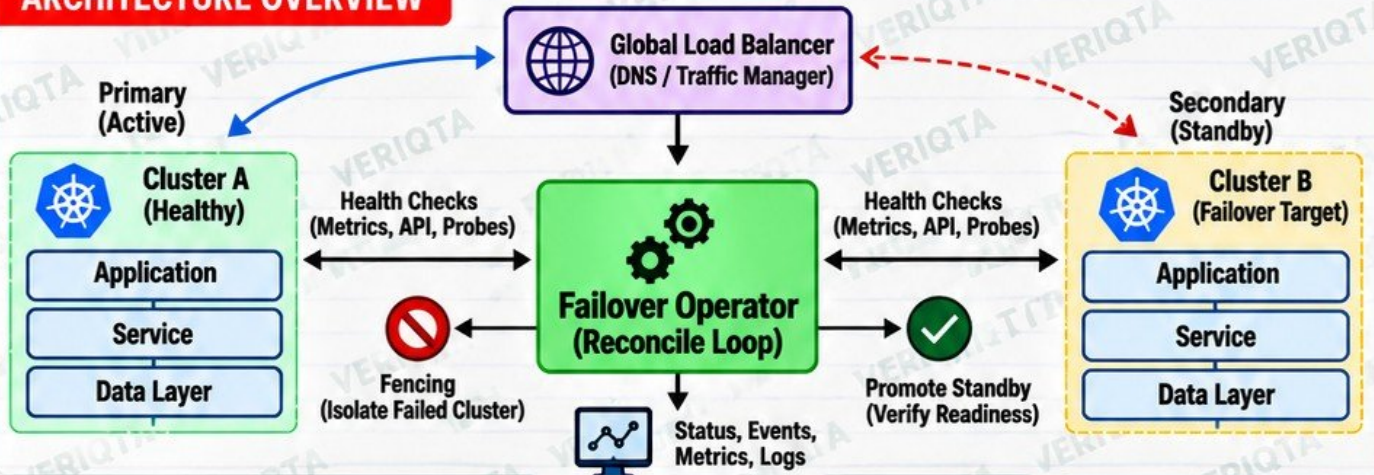
PROJECT OBJECTIVE

- ✓ Monitor health of multiple Kubernetes clusters
- ✓ Detect failures using metrics, probes and API health
- ✓ Coordinate automated traffic failover
- ✓ Enforce fencing to avoid split-brain
- ✓ Verify application readiness before promoting
- ✓ Provide safe failback with approval gates
- ✓ Track status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1
kind: MultiClusterFailover
metadata:
  name: app-failover
  namespace: platform
spec:
  primaryCluster: cluster-a
  secondaryClusters:
    - name: cluster-b
    - name: cluster-c
  healthCheck:
    type: http
    endpoint: /healthz
    interval: 30s
    failureThreshold: 3
  trafficManagement:
    type: dns
    dnsProvider: cloudflare
  fencing:
    enabled: true
    method: isolate-namespace
  promotionCriteria:
    readinessChecks: true
    approvalRequired: false
  failback:
    enabled: true
    strategy: manual
```

API version

Resource type

Resource name

Namespace

Active cluster

Standby clusters

Health check configuration

Traffic management

Fencing configuration

Promotion rules

Failback configuration

RECONCILIATION FLOW

- 1 Monitor cluster health (API, metrics, application probes)
- 2 Detect failure based on thresholds
- 3 Fence failed cluster to prevent split-brain
- 4 Verify standby cluster readiness
- 5 Update DNS / load balancer to route traffic to standby
- 6 Confirm application health after failover
- 7 Update status, events and metrics
- 8 Handle failback (manual or automatic)

PRODUCTION CONSIDERATIONS

- ✓ Use multiple independent health signals
- ✓ Implement fencing to avoid split-brain
- ✓ Support manual approval for critical failover
- ✓ Validate data consistency before promotion
- ✓ Handle partial failures and network partitions
- ✓ Use idempotent reconciliation logic
- ✓ Monitor failover and recovery metrics
- ✓ Test in real multi-cluster environment

FAILURE SCENARIOS

- ! Complete cluster outage
- ! Control plane unreachable
- ! Network partition between clusters
- ! Split-brain risk
- ! DNS or load balancer failure
- ! Standby cluster not ready
- ! Data inconsistency during failover
- ! Continuous failover flapping



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

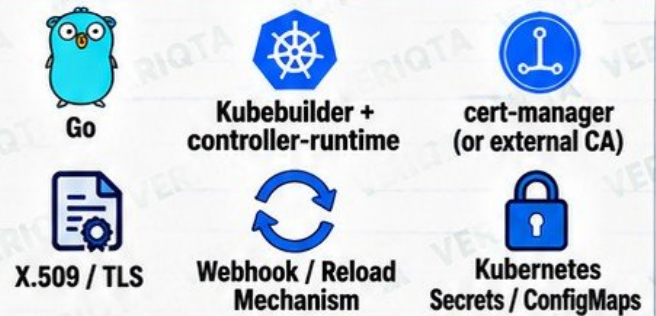
04 Build a Kubernetes Certificate Lifecycle and Rotation Operator

Automate certificate issuance, renewal, trust-bundle overlap, and workload reloads without service interruption.

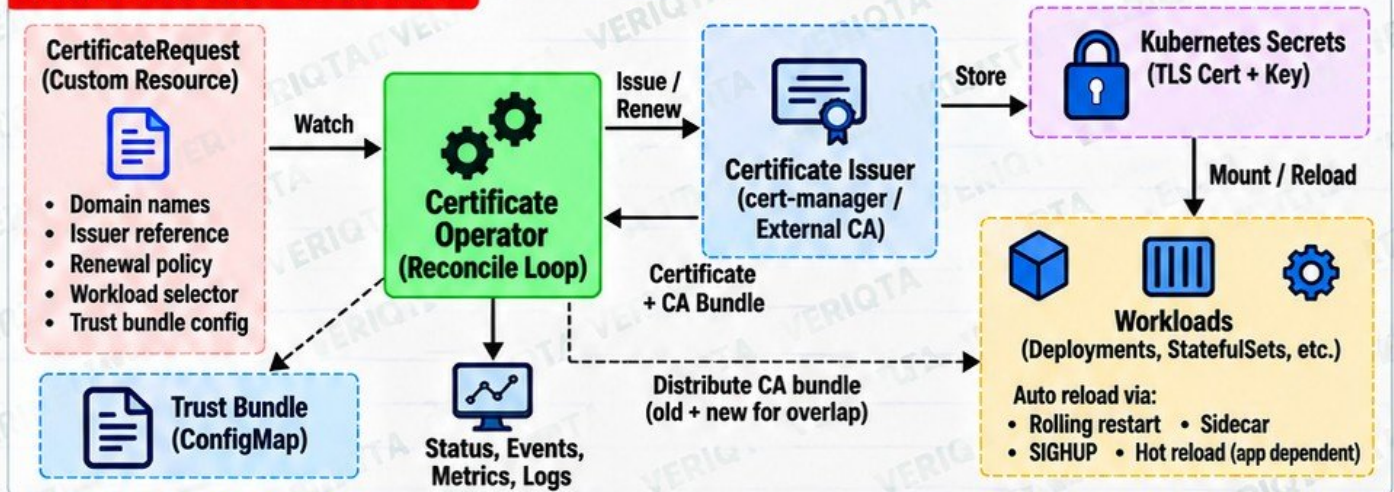
PROJECT OBJECTIVE

- ✓ Automate certificate issuance for workloads
- ✓ Renew certificates before expiry
- ✓ Maintain trust-bundle overlap (old + new)
- ✓ Trigger safe workload reloads without downtime
- ✓ Support multiple issuers (e.g. CA, cert-manager, external PKI)
- ✓ Handle revocation and cleanup
- ✓ Expose status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqa.io/v1 # API version
kind: CertificateRequest # Resource type
metadata:
  name: app-tls # Resource name
  namespace: production # Namespace
spec:
  commonName: api.example.com # Certificate common name
  dnsNames: # Additional SANs
    - api.example.com
    - api.internal.svc.cluster.local
  issuerRef: # Certificate issuer
    name: prod-ca # Reference type
    kind: ClusterIssuer
  renewal: # Renewal configuration
    beforeExpiry: 720h # 30 days # Rotate before expiry
    mode: auto # Renewal mode
  trustBundle: # Trust bundle settings
    enabled: true # Keep old + new CA
    overlapDuration: 168h # 7 days # Overlap duration
  workloadRef: # Target workload
    kind: Deployment
    name: api-service
  reload: # Reload configuration
    strategy: rolling # Reload method
    annotation: "reloader.stakater.com/auto: \"true\""

```

RECONCILIATION FLOW

- 1 Read CertificateRequest and validate configuration
- 2 Check existing certificate and expiry time
- 3 Request new certificate from issuer (if needed)
- 4 Create or update TLS secret
- 5 Update trust bundle (old + new CA for overlap)
- 6 Trigger workload reload using configured strategy
- 7 Update status, events and metrics
- 8 Handle errors with retries and backoff

PRODUCTION CONSIDERATIONS

- ✓ Use short certificate lifetimes (e.g. 30-90 days)
- ✓ Maintain CA bundle overlap to avoid trust issues
- ✓ Support multiple issuers and environments
- ✓ Implement safe reload strategy per workload
- ✓ Protect private keys and restrict RBAC access
- ✓ Monitor expiry, renewal failures and reload status
- ✓ Handle partial failures and ensure idempotency
- ✓ Support revocation and cleanup on deletion

FAILURE SCENARIOS

- ! Certificate issuer unavailable
- ! Renewal failure or rate limit
- ! Workload reload fails or unhealthy
- ! Trust bundle not updated in time
- ! Expired certificate due to missed renew
- ! RBAC permission denied
- ! Secret update conflict
- ! Application cannot reload certificates
- ! Split trust between old and new CA
- ! Operator crash during rotation



@veriqa



@veriqa



@veriqa



@veriqa



@veriqa



@veriqa

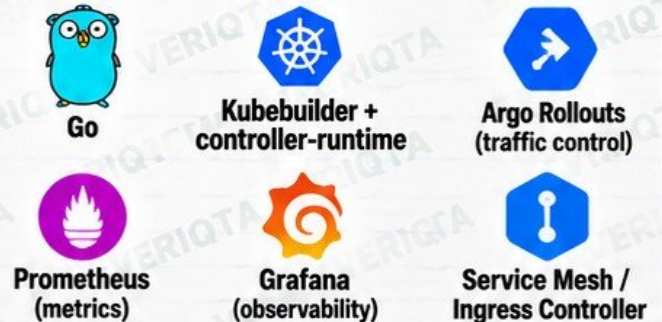
05 Build an SLO-Aware Progressive Delivery Controller

Control canary deployments using latency, error-budget burn rates, traffic weights, and automated rollback decisions.

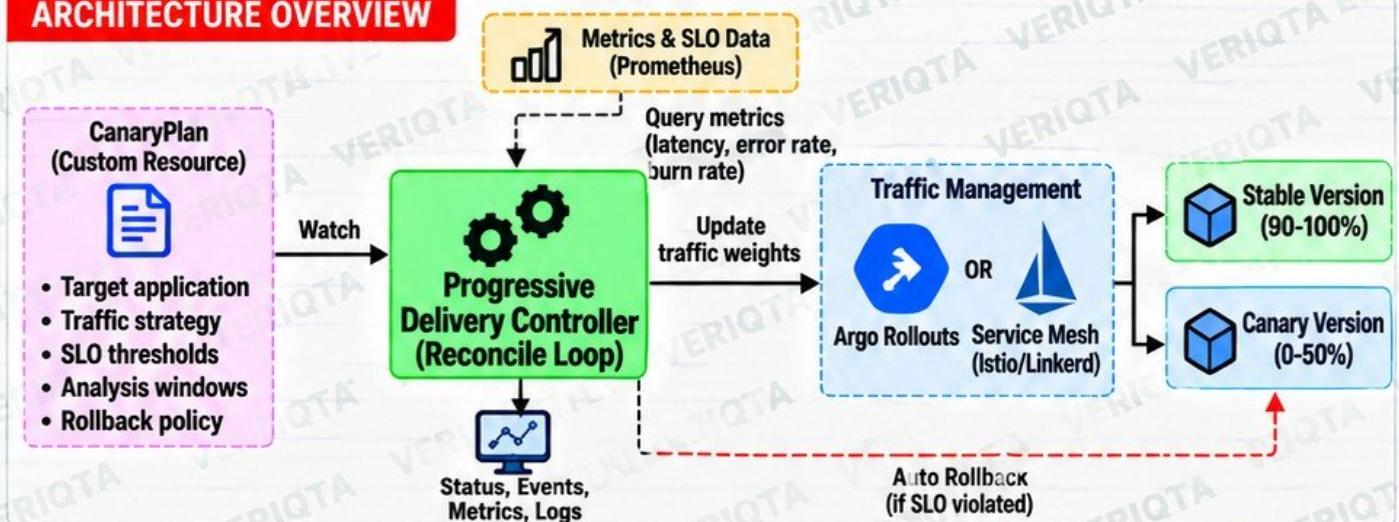
PROJECT OBJECTIVE

- ✓ Implement canary and progressive delivery
- ✓ Use real SLO and error budget signals
- ✓ Control traffic weights automatically
- ✓ Detect issues using latency, error rate and burn rate
- ✓ Automate rollback when SLOs are at risk
- ✓ Support multiple deployment strategies
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1      # API version
kind: CanaryPlan              # Resource type
metadata:
  name: payment-service       # Plan name
  namespace: production       # Namespace
spec:
  targetRef:                  # Target application
    kind: Deployment
    name: payment-service
  strategy: canary             # Deployment to control
  traffic:                    # Canary strategy
    initialWeight: 5           # Traffic configuration
    steps:                    # Start with 5% traffic
      - setWeight: 25          # Increase to 25% and pause
        pause: 10m
      - setWeight: 50          # Increase to 50% and pause
        pause: 10m
      - setWeight: 100         # Increase to 100%
analysis:
  metrics:                    # Traffic configuration
    - name: request_latency_p95 # Start with 5% traffic
      threshold: 300            # ms
    - name: error_rate
      threshold: 1              # %
    - name: error_budget_burn_rate
      threshold: 2              # 2x burn rate
rollback:
  strategy: automatic          # Automatic rollback policy
  timeout: 5m                  # Rollback if metrics violate SLO
                                # Analysis timeout
```

RECONCILIATION FLOW

- 1 Read CanaryPlan and validate configuration
- 2 Check application and service readiness
- 3 Query SLO metrics (latency, error rate, burn rate)
- 4 Evaluate promotion or rollback conditions
- 5 Update traffic weights (Argo Rollouts or Service Mesh)
- 6 Monitor metrics during each step
- 7 Auto rollback if SLO is violated
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use real SLO and error budget policies
- ✓ Set conservative rollout steps and analysis windows
- ✓ Protect against false positives (use multiple signals)
- ✓ Support manual approval for high-risk services
- ✓ Integrate with Argo Rollouts or service mesh
- ✓ Handle metrics outages gracefully
- ✓ Ensure rollback is fast and automated
- ✓ Monitor controller performance and reconcile delays

FAILURE SCENARIOS

- ! Latency increases during canary
- ! Error rate exceeds threshold
- ! Error budget burns too fast
- ! Metrics system unavailable
- ! Canary becomes unhealthy
- ! Traffic weight update fails
- ! Rollback fails or is delayed
- ! Service mesh / ingress misconfiguration
- ! Controller restart during rollout



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

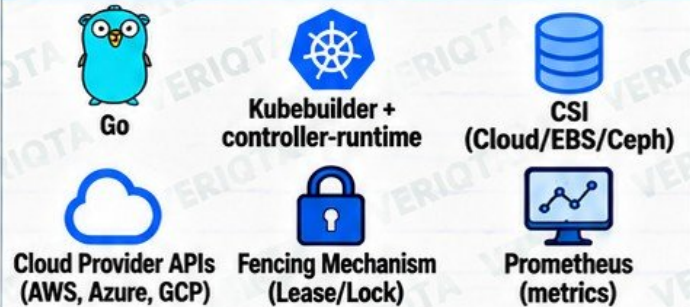
06 Build a Stateful Workload Recovery and Volume Failover Operator

Orchestrate storage recovery, volume attachment safety, workload fencing, and recovery verification after node or zone failure.

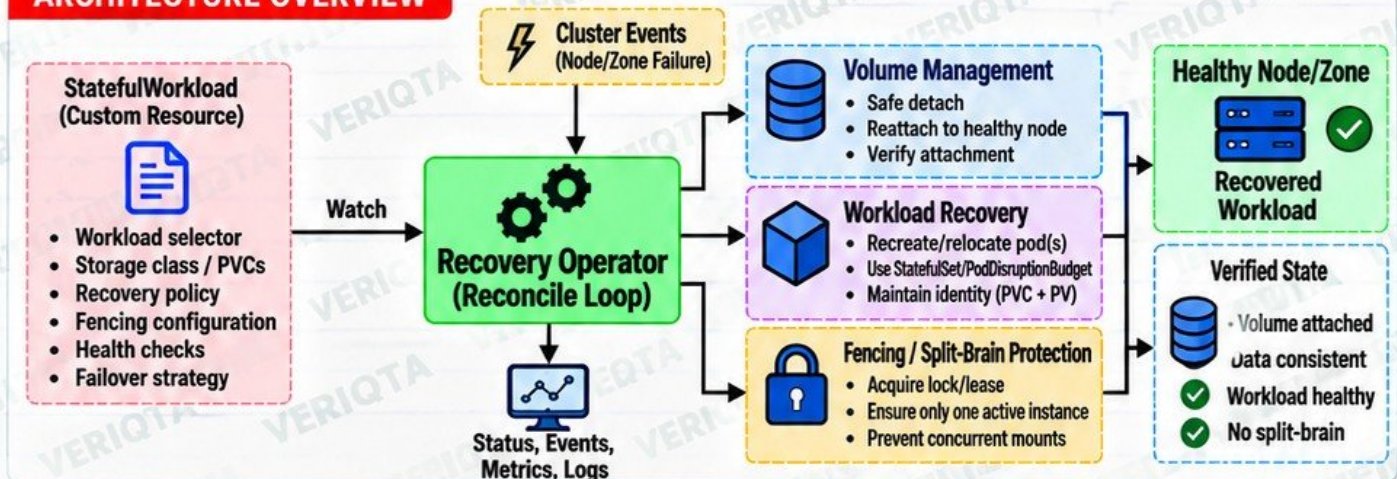
PROJECT OBJECTIVE

- ✓ Detect node or zone failures affecting stateful workloads
- ✓ Ensure safe volume detachment and reattachment
- ✓ Implement fencing to prevent split-brain
- ✓ Automate workload recovery to healthy nodes/zones
- ✓ Verify data and workload health after recovery
- ✓ Support multiple storage backends (e.g. EBS, Ceph, CSI)
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1
kind: StatefulWorkload
metadata:
  name: orders-db
  namespace: production
spec:
  replicas: 1
  selector:
    matchLabels:
      app: orders-db
  storage:
    className: ebs-gp3
    size: 500Gi
  recoveryPolicy:
    strategy: auto
    nodeFailure: retry
    zoneFailure: failover
  fencing:
    enabled: true
    method: lease
  healthCheck:
    type: tcp
    port: 5432
    timeout: 30s
    interval: 10s
```

API version

Resource type

Metadata

Workload name

Namespace

Desired state

Stateful workload

Pod selector

Storage configuration

Storage class

Volume size

Recovery configuration

auto | manual

Node failure action

Zone failure action

Fencing configuration

lease | external-lock

Health check configuration

tcp | http | exec

RECONCILIATION FLOW

- 1 Watch StatefulWorkload and cluster events
- 2 Detect node/zone failure or unhealthy workload
- 3 Acquire fencing lock to prevent split-brain
- 4 Safely detach volumes from failed node (if required)
- 5 Reattach volumes to healthy node/zone
- 6 Recreate or relocate workload using StatefulSet
- 7 Verify volume attachment, data consistency and workload health
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use PodDisruptionBudgets and anti-affinity
- ✓ Handle cloud provider volume attach/detach limits
- ✓ Implement fencing to prevent split-brain
- ✓ Verify data integrity after recovery
- ✓ Support multi-zone and multi-region deployments
- ✓ Implement retries, backoff and idempotent reconciliation
- ✓ Monitor with Prometheus metrics and alerts
- ✓ Test with real node/zone failure scenarios

FAILURE SCENARIOS

- ! Node failure during write operations
- ! Zone outage and volume unavailability
- ! Volume stuck in attaching/detaching
- ! Split-brain with multiple active pods
- ! Data inconsistency after failover
- ! Cloud API rate limits or failures
- ! Stale fencing lock not released
- ! Pod scheduling failures
- ! Recovery verification timeout



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

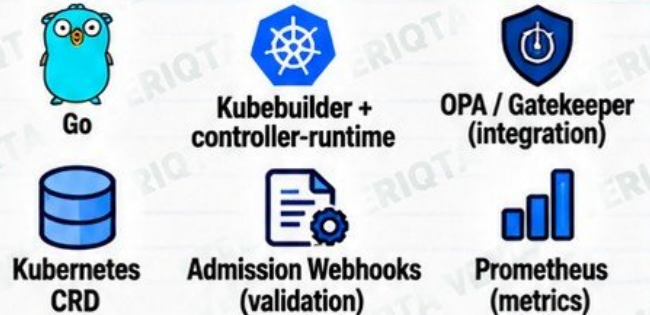
07 Build a Multi-Tenant Resource Governance Operator

Enforce tenant quotas, namespace isolation, resource policies, ownership boundaries, and controlled exception workflows.

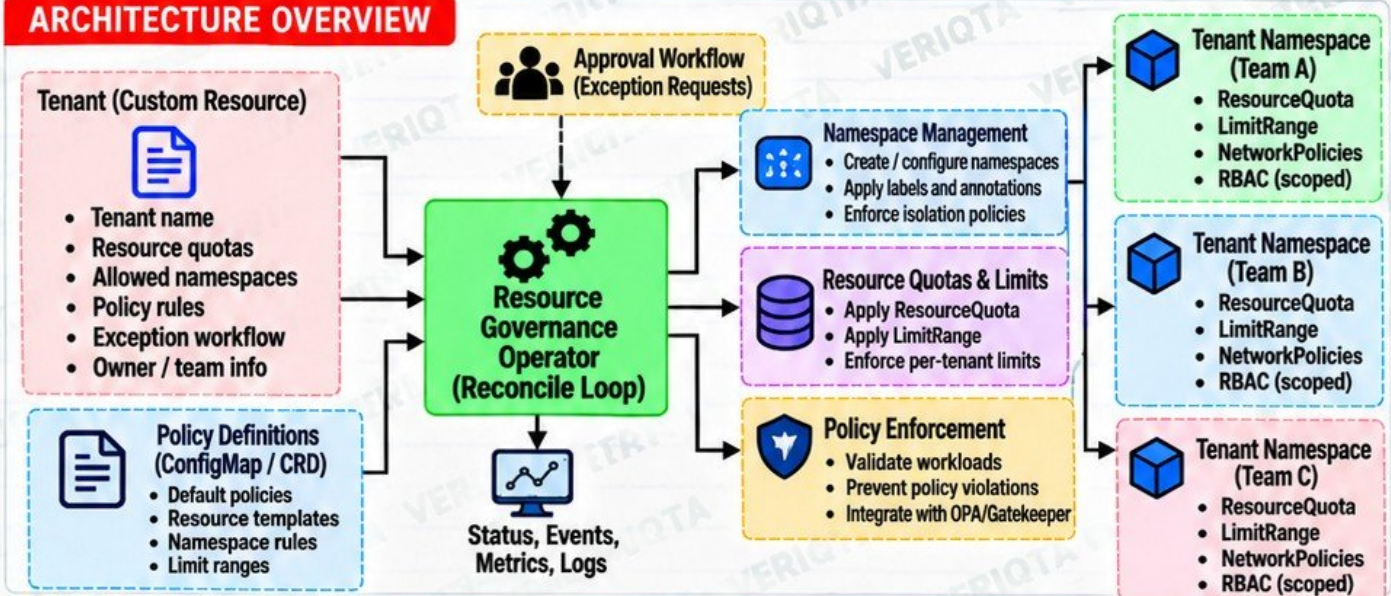
PROJECT OBJECTIVE

- ✓ Enforce tenant quotas and resource limits
- ✓ Implement namespace isolation and policies
- ✓ Apply resource governance rules automatically
- ✓ Manage ownership and team boundaries
- ✓ Handle controlled exception requests and approvals
- ✓ Detect and prevent policy violations
- ✓ Provide clear status, events and audit visibility

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1      # API version
kind: Tenant                  # Resource type
metadata:
  name: team-a                # Tenant name
spec:
  displayName: "Team A"       # Display name
  namespacePrefix: team-a     # Namespace prefix
  resourceQuota:
    cpu: "10"                 # Total CPU limit
    memory: "20Gi"            # Total memory limit
    storage: "100Gi"          # Total storage limit
  limitRange:
    defaultCpu: "500m"        # Default CPU per container
    defaultMemory: "1Gi"      # Default memory per container
  allowedNamespaces:
    - team-a-dev               # Allowed namespaces for tenant
    - team-a-prod
  allowedImages:
    - registry.example.com/*   # Approved container registries
  networkPolicy: enabled      # Enable network isolation
  rbac:
    teamRole: developer        # Role for tenant team
  exceptionPolicy:
    enabled: true              # Enable exception workflow
    approvalRequired: true     # Require approval
    approvers:
      - platform-team          # Approvers for exceptions
```

RECONCILIATION FLOW

- 1 Read Tenant custom resource and validate configuration
- 2 Create or update namespaces with required labels
- 3 Apply ResourceQuota, LimitRange and NetworkPolicies
- 4 Configure RBAC and ownership boundaries
- 5 Validate policies using admission webhooks / OPA
- 6 Handle exception requests and approvals
- 7 Detect and remediate policy violations
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Enforce strict namespace and resource isolation
- ✓ Use admission webhook validation
- ✓ Integrate with OPA/Gatekeeper for policy control
- ✓ Implement exception approval workflow
- ✓ Support multi-team and multi-environment setup
- ✓ Provide audit logs and metrics for compliance
- ✓ Handle policy updates without disrupting workloads
- ✓ Test with real-world policy violation scenarios

FAILURE SCENARIOS

- ❗ Tenant exceeds resource quota
- ❗ Unauthorized namespace creation
- ❗ Policy violation (restricted image)
- ❗ RBAC escalation attempt
- ❗ Network policy misconfiguration
- ❗ Exception request without approval
- ❗ Operator crash during reconciliation
- ❗ Stale policy after update
- ❗ Conflicting policies between teams



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

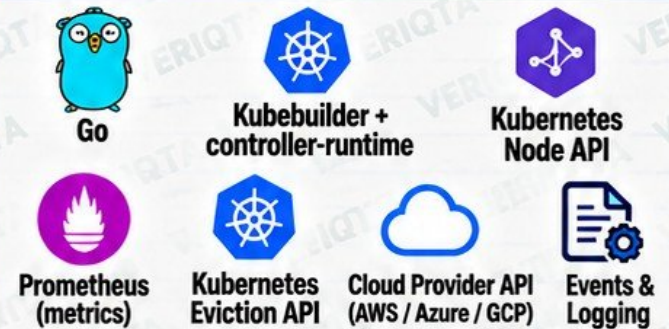
08 Build a Kubernetes Node Remediation Controller

Detect unhealthy nodes, distinguish transient faults from sustained failures, coordinate safe draining, and enforce remediation limits.

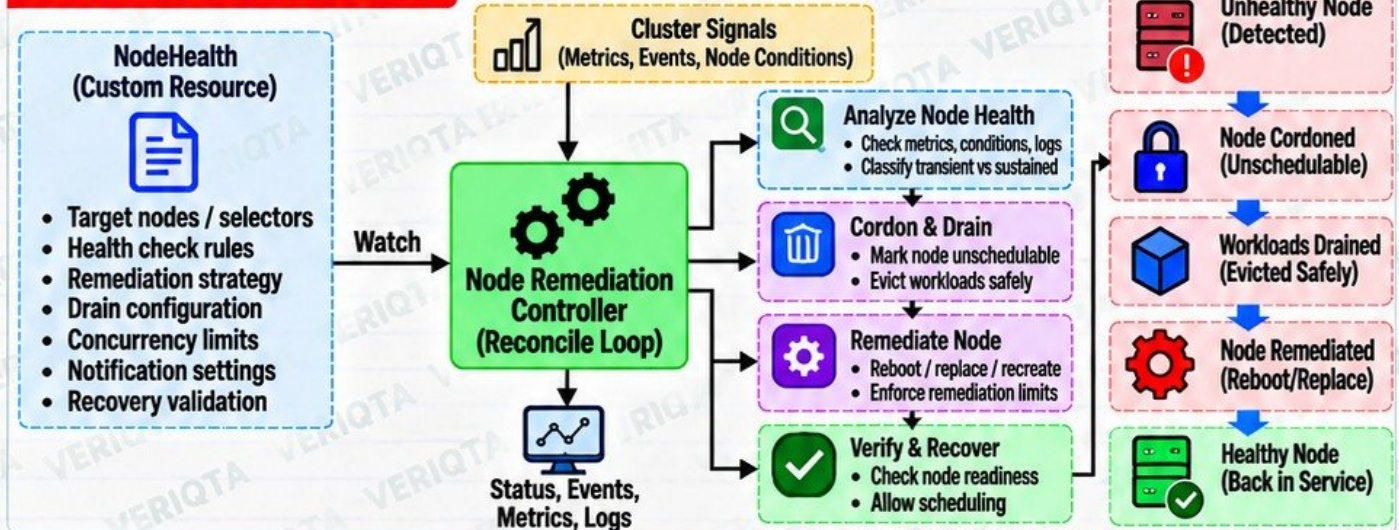
PROJECT OBJECTIVE

- ✓ Detect unhealthy nodes using multiple signals
- ✓ Distinguish transient faults from sustained failures
- ✓ Coordinate safe cordon and drain operations
- ✓ Trigger automated remediation (replace/reboot/recreate)
- ✓ Enforce remediation limits to prevent disruption
- ✓ Verify node recovery and workload rescheduling
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1 # API version
kind: NodeHealth           # Resource type
metadata:
  name: worker-nodes       # Resource name
  namespace: platform      # Namespace
spec:
  nodeSelector:            # Node selection
    matchLabels:
      node-role.kubernetes.io/worker: ""
  healthChecks:            # Health check rules
    - type: nodeCondition  # Condition-based check
      conditions: [Ready, DiskPressure, MemoryPressure] # Metrics-based check
      duration: 5m
    - type: metrics        # Notification settings
      cpuUsageThreshold: 90
      memoryUsageThreshold: 90
      duration: 10m
  remediation:
    strategy: replace      # options: reboot | replace | recreate
    maxConcurrent: 1       # Max nodes to remediate at once
    retryLimit: 3          # Retry count for failed remediation
  drain:
    gracePeriod: 300       # seconds
    ignoreDaemonSets: true
    deleteEmptyDirData: false
  notifications:
    events: true
    slackWebhook: "https://hooks.slack.com/..."
```

RECONCILIATION FLOW

- 1 Read NodeHealth and validate configuration
- 2 Collect node health signals (conditions, metrics, logs)
- 3 Classify node issue (transient or sustained)
- 4 Enforce remediation limits and concurrency rules
- 5 Cordon node if remediation is required
- 6 Drain workloads using Kubernetes eviction API
- 7 Perform remediation (reboot / replace / recreate)
- 8 Verify node recovery and workload rescheduling
- 9 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use multiple health signals and thresholds
- ✓ Avoid false positives (transient faults)
- ✓ Graceful and safe workload draining
- ✓ Enforce remediation limits and rate controls
- ✓ Support different remediation strategies
- ✓ Integrate with cloud provider APIs
- ✓ Provide clear alerting and notifications
- ✓ Test with real node failure scenarios

FAILURE SCENARIOS

- ❗ Transient network or kubelet failure
- ❗ High CPU or memory pressure
- ❗ Disk or inode exhaustion
- ❗ Node NotReady / Unknown state
- ❗ Unresponsive node (no heartbeat)
- ❗ Eviction API failure or stuck pods
- ❗ Cloud instance remediation failure
- ❗ Repeated remediation cycles
- ❗ Split-brain after node recovery



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

09 Build an External Secrets and Workload Identity Rotation Operator

Synchronize external credentials, coordinate rotation windows, enforce workload identity, and verify successful secret consumption.

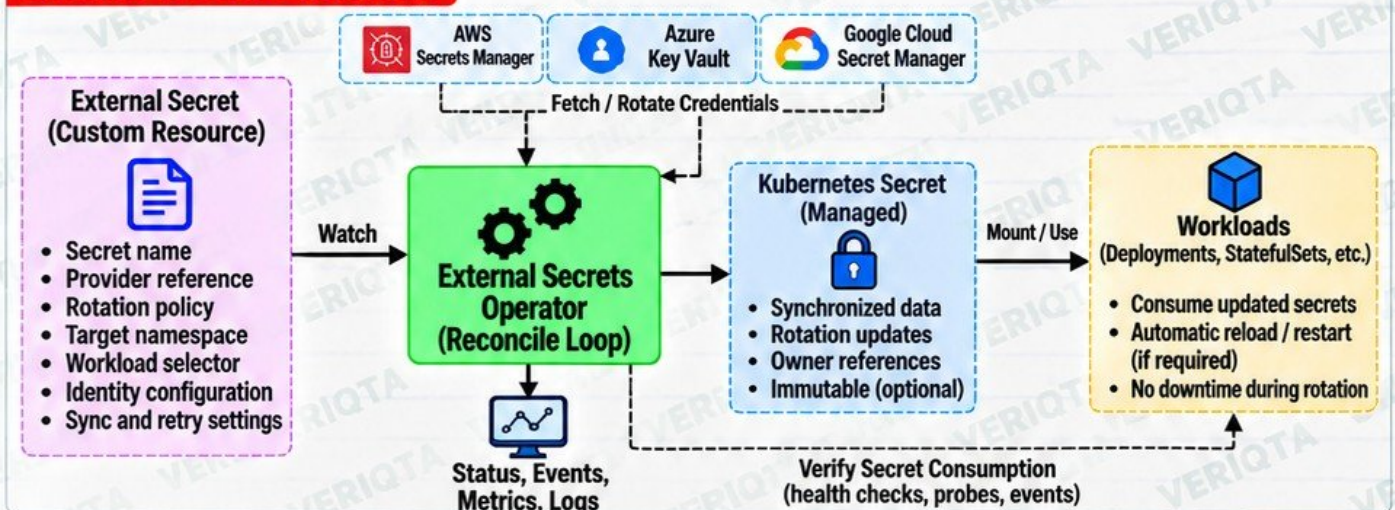
PROJECT OBJECTIVE

- ✓ Synchronize secrets from external providers
- ✓ Automate credential rotation and renewal
- ✓ Enforce workload identity (IAM/IRSA/Workload Identity)
- ✓ Coordinate safe rotation windows
- ✓ Validate secret consumption by workloads
- ✓ Handle failures, rollback and alerting
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1
kind: ExternalSecret
metadata:
  name: app-database
  namespace: production
spec:
  provider:
    name: aws-secrets-manager
    region: us-east-1
    secretRef: prod/db
  target:
    name: app-db-secret
    namespace: production
  rotation:
    interval: 24h
    window: 02:00-04:00
    strategy: rolling
  workloadIdentity:
    enabled: true
    provider: aws
    serviceAccount: app-sa
  validation:
    checkConsumption: true
    healthEndpoint: /health
  retryPolicy:
    maxRetries: 3
    backoff: 1m
```

API version
Resource type
Resource name
Namespace
Secret provider
Provider region
External secret path
Target Kubernetes secret
Target namespace
Rotate every 24 hours
Rotation window (UTC)
Rotation strategy
Enable workload identity
Provider type
Service account to use
Verify secret consumption
Health check path
Retry configuration

RECONCILIATION FLOW

- 1 Read ExternalSecret and validate configuration
- 2 Authenticate to external provider using workload identity
- 3 Fetch or rotate credentials (if required)
- 4 Create or update Kubernetes Secret
- 5 Verify secret data and permissions
- 6 Ensure workloads consume the updated secret
- 7 Handle failures, retries and rollback
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use workload identity (no static cloud keys)
- ✓ Secure RBAC and namespace isolation
- ✓ Enforce rotation windows and rate limits
- ✓ Validate secret consumption automatically
- ✓ Minimize downtime during rotation
- ✓ Handle provider API failures and throttling
- ✓ Audit access and changes
- ✓ Monitor with metrics, alerts and events

FAILURE SCENARIOS

- ! External provider API unavailable
- ! Invalid or expired credentials
- ! Permission/identity misconfiguration
- ! Secret rotation fails or partial update
- ! Workload cannot consume new secret
- ! Secret out of sync with external store
- ! Rate limit exceeded
- ! Rollback to previous secret version
- ! Operator crash during rotation



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

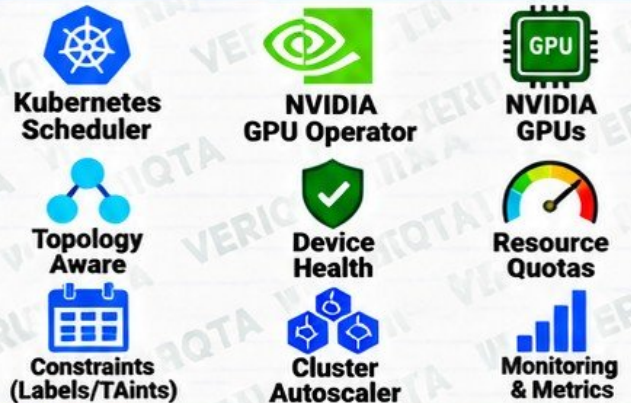
10 Build a GPU Workload Placement and Capacity Operator

Allocate GPU capacity using topology awareness, device health, quotas, scheduling constraints, and fragmentation signals.

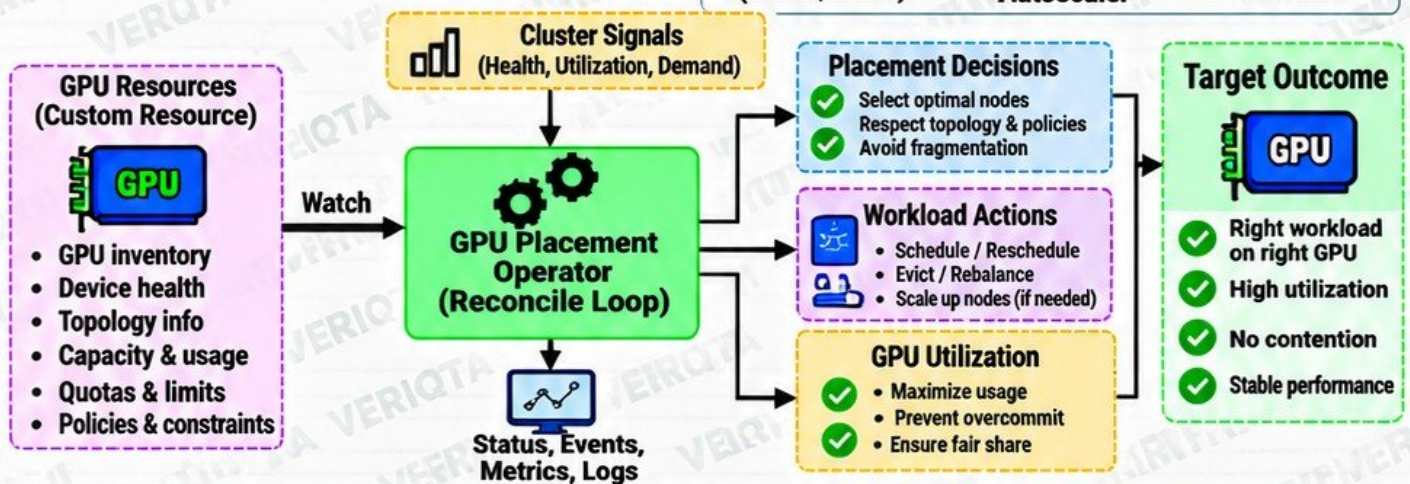
PROJECT OBJECTIVE

- ✓ Smart GPU placement using topology
- ✓ Respect device health and availability
- ✓ Enforce quotas and policies
- ✓ Honor scheduling constraints
- ✓ Reduce fragmentation and waste
- ✓ Maximize utilization and performance
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: v1
kind: GPUWorkload
metadata:
  name: ai-training-job
spec:
  replicas: 4
  gpu:
    type: nvidia.com/gpu
    count: 1
  constraints:
    topology.kubernetes.io/zone: ["zone-a"]
    nvidia.com/gpu-memory: ">= 24Gi"
  policy:
    maxUtilization: 90
    avoidFragmentation: true
    preferHealthyNodes: true
```

RECONCILIATION FLOW

- 1 Read GPUWorkload and cluster state
- 2 Check device health and availability
- 3 Evaluate topology, quotas and constraints
- 4 Select optimal nodes for placement
- 5 Schedule / reschedule / evict if needed
- 6 Update status and metrics
- 7 Monitor and prevent fragmentation
- 8 Report events and progress

PRODUCTION CONSIDERATIONS

- ✓ Use device plugins and health checks
- ✓ Set proper quotas and limits
- ✓ Monitor GPU utilization & alerts
- ✓ Handle fragmentation proactively
- ✓ Test with real GPU workloads

FAILURE SCENARIOS

- ! GPU node becomes unhealthy
- ! GPU shortage / high demand
- ! Fragmentation prevents placement
- ! Scheduler conflicts
- ! Device plugin issues



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

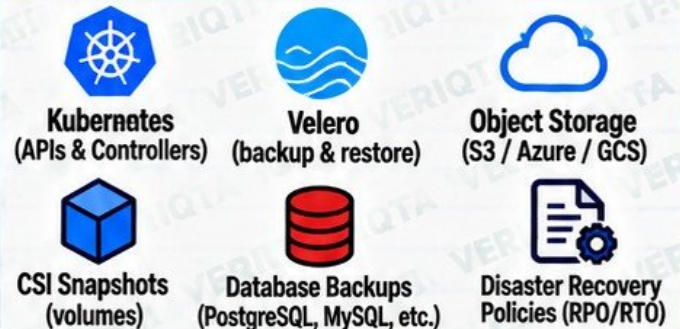
11 Build a Kubernetes Disaster Recovery Orchestration Operator

Coordinate application-consistent backups, dependency-aware restoration, recovery sequencing, and RPO/RTO validation.

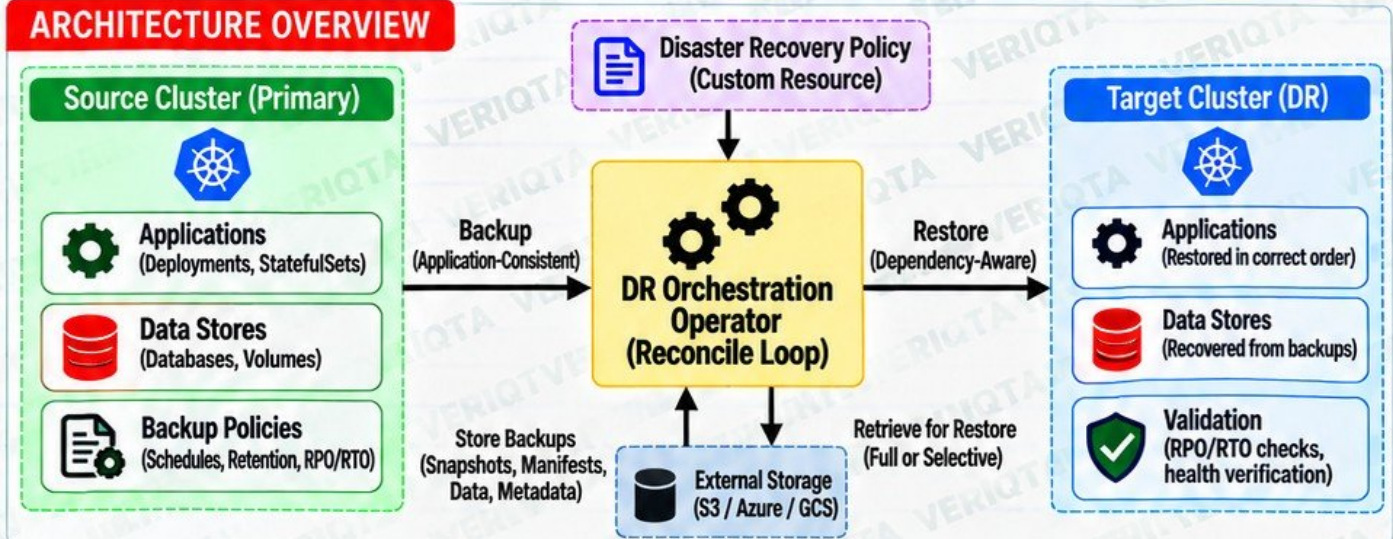
PROJECT OBJECTIVE

- ✓ Automate application-consistent backups
- ✓ Understand and manage application dependencies
- ✓ Coordinate safe and ordered restoration
- ✓ Validate RPO and RTO automatically
- ✓ Handle cross-cluster and cross-region recovery
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqlta.io/v1 # API version
kind: DRPolicy # Resource type
metadata:
  name: production-dr # Policy name
  namespace: platform # Namespace
spec:
  schedule: # Backup schedule
    backupInterval: 6h # Backup every 6 hours
    retention: 30d # Keep backups for 30 days
    storage:
      provider: aws # Object storage provider
      bucket: veriqlta-backups # Backup bucket name
  rpo: "15m" # Recovery point objective
  rto: "1h" # Recovery time objective
  includeNamespaces: # Namespaces to include
    - app-team
    - data-team
  dependencies: # Application dependencies
    - name: postgres
      type: database # PostgreSQL backup
      backup: enabled # Restergeioim t oncturds
    - name: redis
      type: statefulset # Redis backup
      backup: enabled
  restoreOrder: # Restoration order
    - postgres
    - redis
    - app-team
```

RECONCILIATION FLOW

- 1 Read DRPolicy and validate configuration
- 2 Trigger application-consistent backups
- 3 Capture manifests, volumes and database backups
- 4 Store backups in external storage
- 5 Monitor backup status and verify integrity
- 6 When required, orchestrate restoration
- 7 Restore dependencies in correct sequence
- 8 Validate application health and RPO/RTO
- 9 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Ensure application-consistent backups
- ✓ Include dependency mapping
- ✓ Use immutable and versioned storage
- ✓ Encrypt backups in transit and at rest
- ✓ Automate regular restore tests
- ✓ Validate RPO and RTO automatically
- ✓ Support cross-region / cross-cluster recovery
- ✓ Provide clear alerting and notifications

FAILURE SCENARIOS

- ! Backup job fails or is incomplete
- ! Corrupted or missing backup data
- ! Dependency restore order failure
- ! Application fails health checks
- ! RPO or RTO not met
- ! Storage provider unavailable
- ! Cross-region network failure
- ! Restored application inconsistent
- ! Operator crash during restore



@veriqlta



@veriqlta



@veriqlta



@veriqlta



@veriqlta



@veriqlta

12 Build an Event-Driven Workload Scaling Controller

Scale consumers from queue lag, processing latency, throughput, and downstream capacity while preventing oscillation.

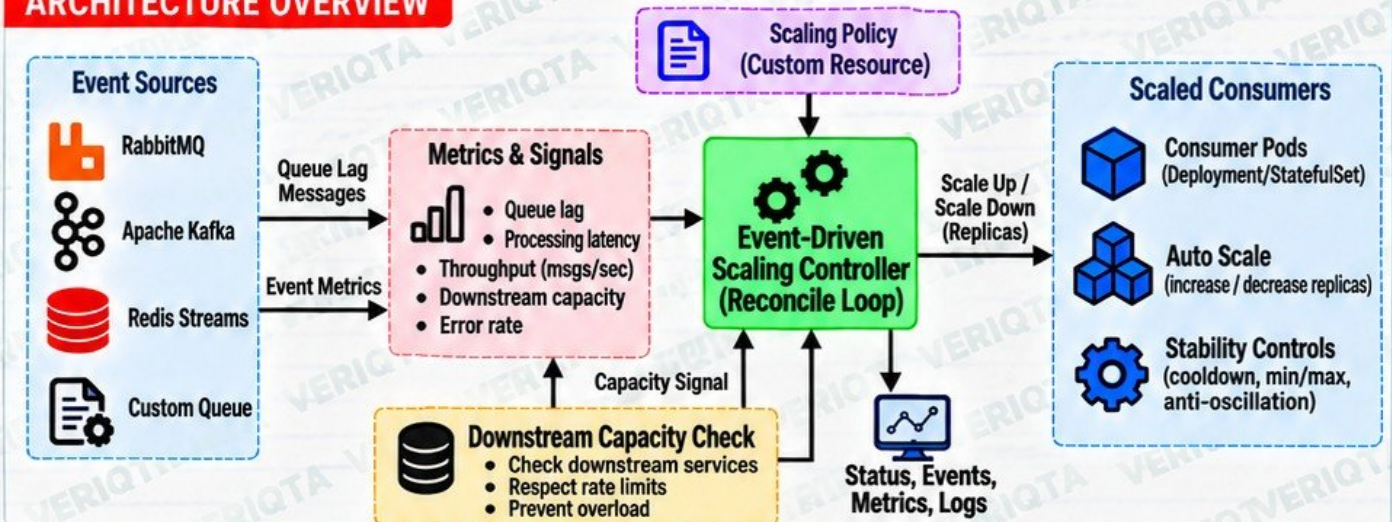
PROJECT OBJECTIVE

- ✓ Scale consumers based on real-time signals
- ✓ Use queue lag, processing latency and throughput
- ✓ Consider downstream capacity and limits
- ✓ Prevent rapid scaling and oscillation (stability)
- ✓ Support scale up and scale down policies
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1 # API version
kind: EventScalePolicy # Resource type
metadata:
  name: queue-consumer-scaling # Resource name
  namespace: production # Namespace
spec:
  source: # Event source configuration
    type: kafka
    topic: orders
  scaling: # Minimum replicas, Maximum replicas, Scaling metrics
    minReplicas: 2
    maxReplicas: 50
    metrics:
      - type: queueLag # Queue lag threshold
        target: 1000
      - type: processingLatency # Latency threshold
        target: "30s"
      - type: throughput # Throughput target
        target: 500
  stabilization: # Anti-oscillation settings
    cooldownSeconds: 300 # 5 minutes cooldown
    scaleUpStep: 5 # Replicas to add at a time
    scaleDownStep: 2 # Replicas to remove at a time
  downstream:
    checkUrl: https://api.example.com/health # Downstream capacity check
    maxRps: 1000 # Downstream service endpoint
    # Maximum allowed RPS
```

RECONCILIATION FLOW

- 1 Read EventScalePolicy and validate configuration
- 2 Collect metrics (queue lag, latency, throughput, downstream capacity)
- 3 Evaluate scaling decision using defined rules
- 4 Apply stability controls (cooldown, min/max, anti-oscillation)
- 5 Scale consumers (increase or decrease replicas)
- 6 Verify consumer health and processing
- 7 Monitor downstream capacity and rate limits
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use reliable and durable message queues
- ✓ Monitor queue lag and consumer health
- ✓ Implement anti-oscillation and cooldown
- ✓ Respect downstream capacity and rate limits
- ✓ Handle poison messages and retries
- ✓ Set appropriate min/max replicas
- ✓ Provide detailed metrics and alerting
- ✓ Test with real workload patterns

FAILURE SCENARIOS

- ! Sudden spike in queue lag
- ! Downstream service rate limit exceeded
- ! Consumers crash or unhealthy
- ! Metrics unavailable or delayed
- ! Rapid scaling causing oscillation
- ! Message processing failures (poison messages)
- ! Queue backlog grows unexpectedly
- ! Downstream outage
- ! Scaling stuck at max or min replicas

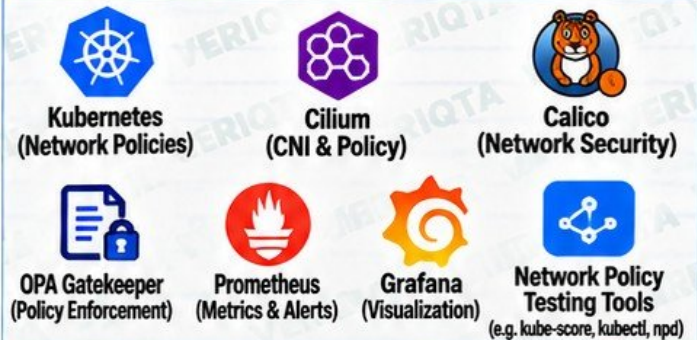
13 Build a Kubernetes Network Isolation and Policy Verification Operator

Generate tenant network policies, detect unsafe connectivity, validate intended access, and prevent policy drift.

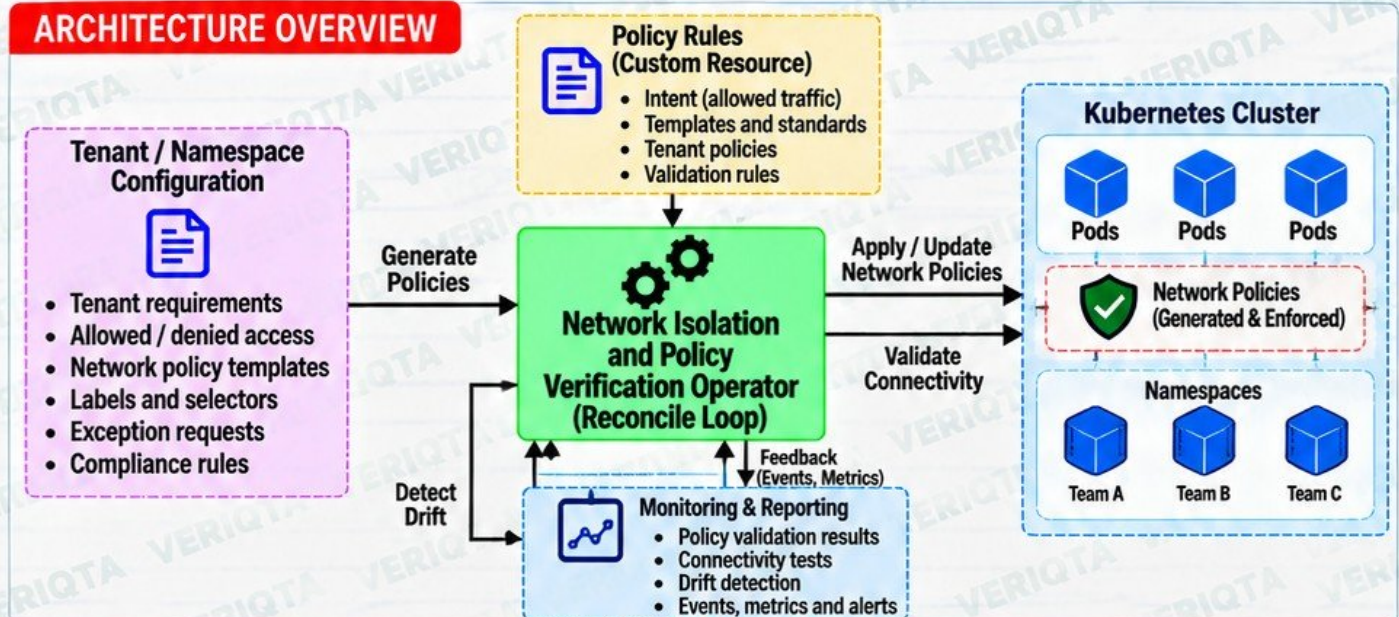
PROJECT OBJECTIVE

- ✓ Generate network policies for tenants and workloads
- ✓ Detect unsafe or unintended connectivity
- ✓ Validate intended access with automated tests
- ✓ Prevent policy drift and enforce compliance
- ✓ Handle alerts, exceptions and remediation
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1
kind: NetworkPolicyConfig
metadata:
  name: tenant-a-policy
  namespace: team-a
spec:
  tenant: team-a
  namespaceSelector:
    matchLabels:
      team: team-a
  allowedIngress:
    - from:
        namespaceSelector:
          matchLabels:
            team: frontend
      ports:
        - protocol: TCP
          port: 8080
  deniedEgress:
    - to: {}
  enforcement: true
  validation:
    testConnectivity: true
    alertOnDrift: true
```

RECONCILIATION FLOW

- 1 Read NetworkPolicyConfig and validate configuration
- 2 Generate or update Kubernetes Network Policies
- 3 Apply policies to target namespaces
- 4 Run connectivity tests (allowed and denied paths)
- 5 Detect unsafe or unintended connectivity
- 6 Check for policy drift and report discrepancies
- 7 Handle tenant exceptions and approvals (if required)
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use a proven CNI (Cilium or Calico)
- ✓ Define clear network policy standards
- ✓ Validate policies with automated tests
- ✓ Monitor for unexpected connectivity
- ✓ Alert on policy drift and violations
- ✓ Support tenant exception workflows
- ✓ Provide detailed audit logs and metrics
- ✓ Test in staging with real workloads

FAILURE SCENARIOS

- ! Unintended cross-namespace access
- ! Policy blocks required traffic
- ! CNI policy not applied correctly
- ! Connectivity test false positives
- ! Policy drift due to manual changes
- ! Tenant misconfiguration
- ! Network policy controller failure
- ! Excessive alerts or noise
- ! Loss of network observability



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

14 Build a Production Configuration Drift and Remediation Controller

Detect divergence from approved configuration, classify drift, enforce ownership boundaries, and apply controlled remediation.

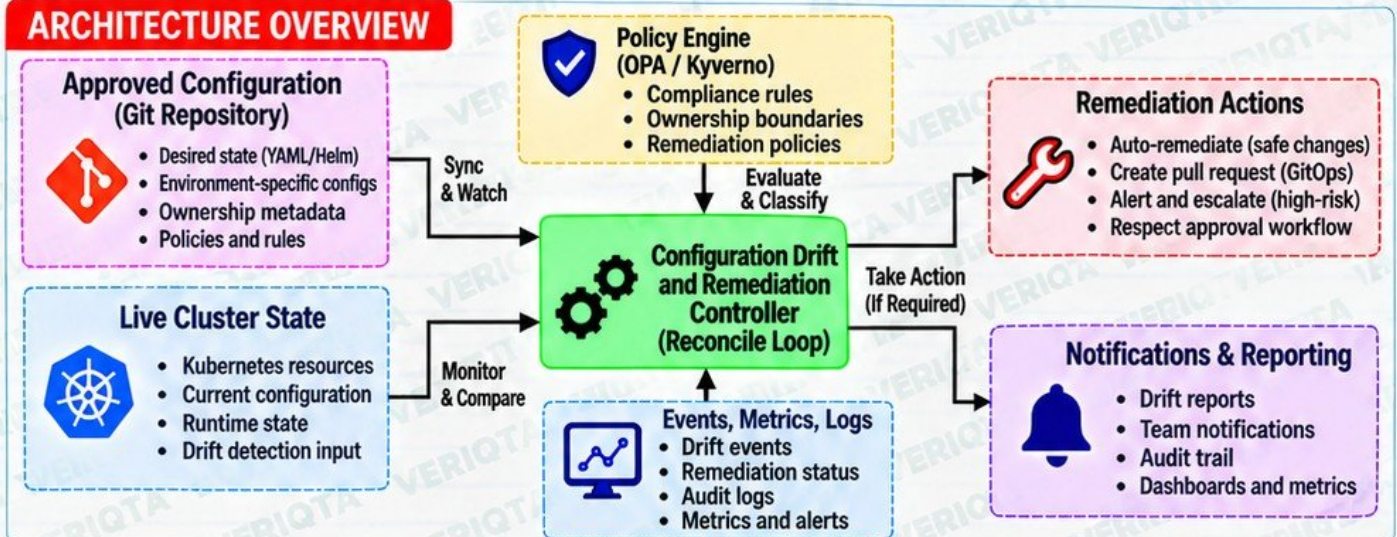
PROJECT OBJECTIVE

- ✓ Detect configuration drift in Kubernetes resources
- ✓ Compare live state with approved configuration
- ✓ Classify drift (intended, unintended, high-risk)
- ✓ Enforce ownership boundaries (teams, namespaces)
- ✓ Apply controlled and safe remediation
- ✓ Provide clear status, events, audit logs and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```

apiVersion: veriqta.io/v1      # API version
kind: ConfigDriftPolicy        # Resource type
metadata:
  name: team-production        # Policy name
  namespace: platform          # Namespace
spec:
  scope:
    namespaces:                # Target namespaces
    - team-a
    - team-b
  resourceTypes:               # Kubernetes resources to monitor
    - configmaps
    - secrets
    - deployments
  policy:
    mode: enforce              # Enforcement mode
    classification:            # Remediation strategy
      highRisk: manual-approval
      mediumRisk: auto-remediate
      lowRisk: report-only
    ownership:                 # Ownership boundary
      team: platform-team
  alerting:
    enabled: true              # Alert configuration
    channel: slack
  metrics:
    enabled: true              # Metrics configuration
  
```

RECONCILIATION FLOW

- 1 Watch live cluster resources and fetch desired configuration
- 2 Compare current state with approved state
- 3 Classify drift (intended / unintended / high-risk)
- 4 Check ownership boundaries and policies
- 5 Apply remediation (auto-fix, PR, or alert)
- 6 Update resource status and events
- 7 Record metrics and audit logs
- 8 Notify teams and enforce compliance

PRODUCTION CONSIDERATIONS

- ✓ Define clear ownership and boundaries
- ✓ Use GitOps for desired state management
- ✓ Classify drift and handle high-risk changes safely
- ✓ Implement approval workflows for sensitive resources
- ✓ Monitor and alert on repeated drift
- ✓ Provide detailed audit logs and metrics
- ✓ Test remediation in non-production first
- ✓ Ensure rollback capability

FAILURE SCENARIOS

- ❗ Unauthorized configuration change
- ❗ Critical resource drift goes undetected
- ❗ Incorrect auto-remediation
- ❗ Policy violation not enforced
- ❗ Approval workflow failure
- ❗ Persistent drift due to permission issues
- ❗ Controller crash or reconciliation failure
- ❗ False positive drift detection



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

15 Build a Kubernetes Dependency-Aware Deployment Controller

Orchestrate service deployments using dependency graphs, readiness contracts, failure propagation, and safe release sequencing.

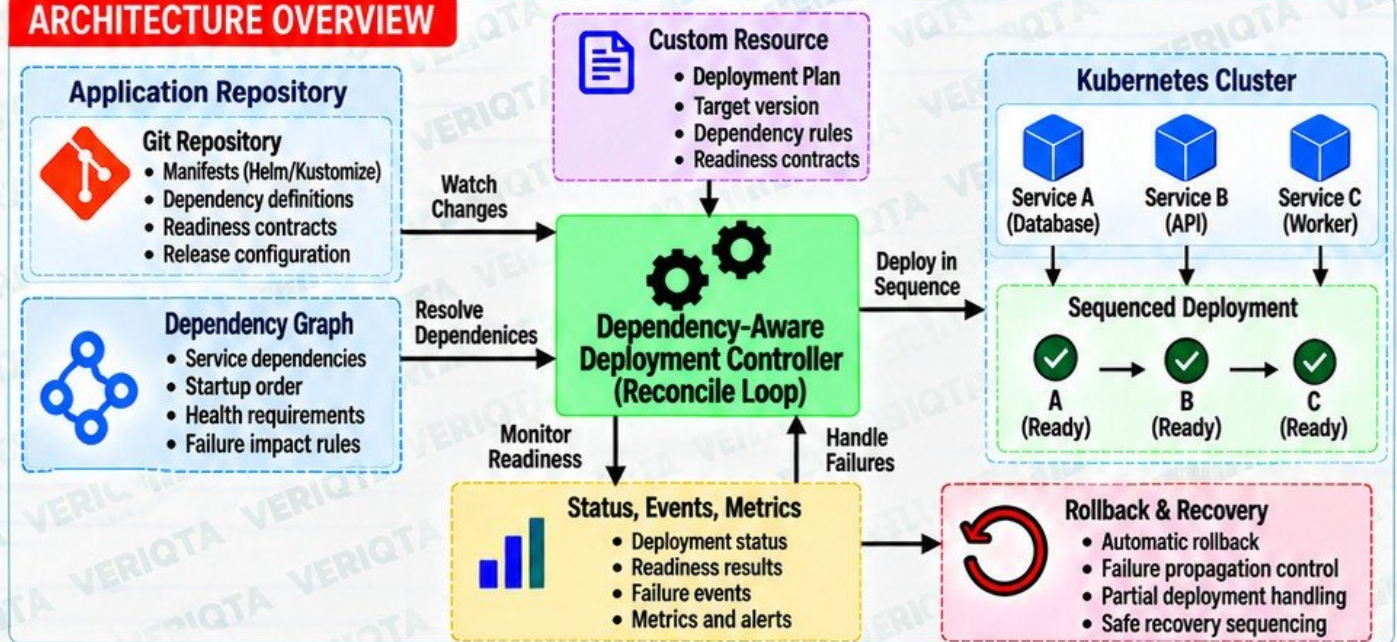
PROJECT OBJECTIVE

- ✓ Deploy services using dependency graphs
- ✓ Honor readiness contracts and health checks
- ✓ Manage deployment sequencing and rollouts
- ✓ Handle failures and prevent cascading impact
- ✓ Support rollback, canary and progressive releases
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```

apiVersion: veriqta.io/v1 # API version
kind: DeploymentPlan      # Resource type
metadata:                 # Metadata
  name: web-app-deployment # Deployment plan name
  namespace: production    # Target namespace
spec:                     # Specification
  version: v1.2.0         # Application version
  services:               # List of services
    - name: database
      version: "14.2"
      readiness:
        type: healthCheck
        endpoint: /ready
    - name: api
      version: "2.0.1"
      dependsOn: ["database"]
      readiness:
        type: http
        endpoint: /health
    - name: worker
      version: "3.1.0"
      dependsOn: ["api"]
      readiness:
        type: queueCheck
        queue: jobs
  releaseStrategy:         # Release strategy
    type: progressive
    maxUnavailable: 1
    rollbackOnFailure: true
  
```

RECONCILIATION FLOW

- 1 Read DeploymentPlan and validate configuration
- 2 Build and validate dependency graph
- 3 Check readiness contracts and health requirements
- 4 Deploy services in correct sequence
- 5 Monitor readiness and verify dependencies
- 6 Handle failures and prevent cascading impact
- 7 Execute rollback or recovery if required
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use clear and versioned dependency definitions
- ✓ Validate readiness contracts before promotion
- ✓ Implement progressive delivery (canary/blue-green)
- ✓ Monitor deployment health and dependency status
- ✓ Set sensible timeouts and failure thresholds
- ✓ Prevent cascading failures with dependency checks
- ✓ Provide detailed audit logs and notifications
- ✓ Test rollback and failure scenarios regularly

FAILURE SCENARIOS

- ! Dependency service fails to become ready
- ! Incorrect dependency order causes outage
- ! Readiness check gives false positive
- ! Partial deployment leaves system unstable
- ! Rollback fails due to configuration drift
- ! Orphaned resources after failed deployment
- ! Progressive rollout stuck or stalled
- ! Controller crash during deployment



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

16 Build a Cloud Infrastructure Lifecycle Operator

Manage cloud resources through Kubernetes custom resources with idempotent provisioning, deletion safeguards, and external-state reconciliation.

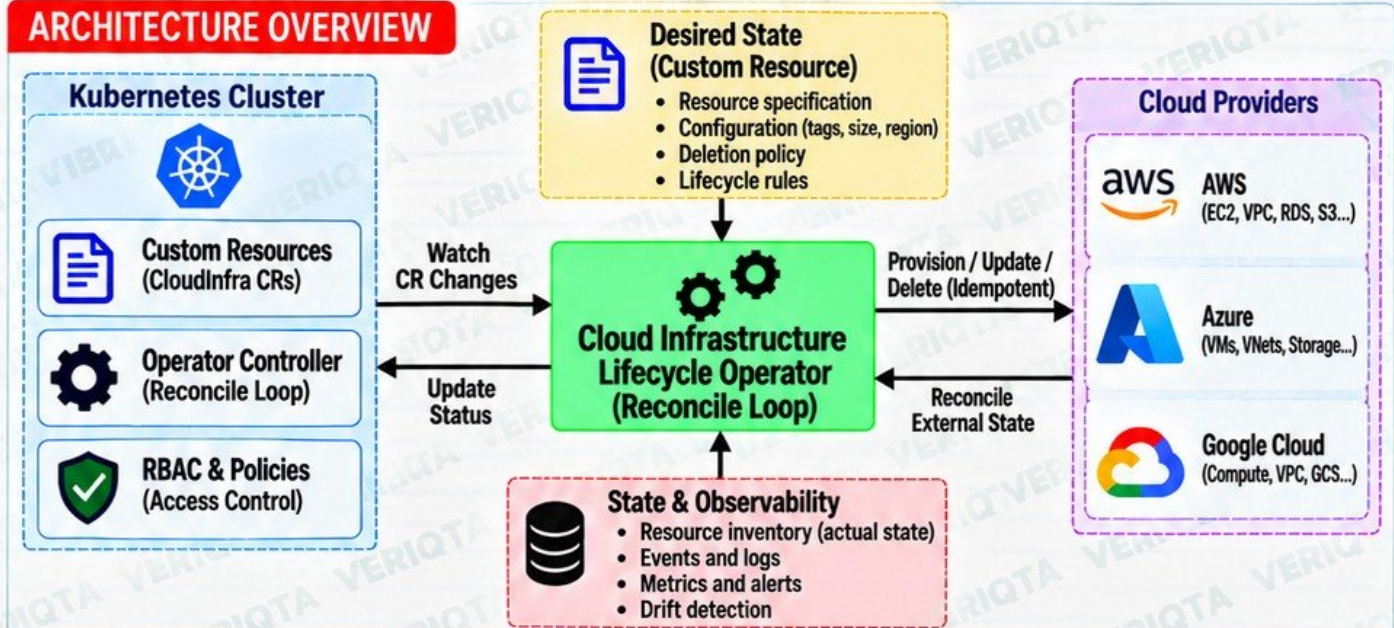
PROJECT OBJECTIVE

- ✓ Manage cloud resources using Kubernetes CRs
- ✓ Ensure idempotent provisioning and updates
- ✓ Implement deletion safeguards and approval workflows
- ✓ Reconcile external cloud state with cluster state
- ✓ Handle failures, retries and drift correction
- ✓ Provide clear status, events and audit logs

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1 # API version
kind: CloudInfrastructure # Resource type
metadata: # Metadata
  name: production-vpc # Resource name
  namespace: platform # Namespace
spec: # Spec
  provider: aws # Cloud provider (aws, azure, gcp)
  resourceType: VPC # Type of resource (VPC, VM, DB, etc)
  region: us-east-1 # Deployment region
  configuration: # Resource configuration
    cidrBlock: "10.0.0.0/16" # CIDR block for the VPC
    enableDnsSupport: true # Enable DNS support
    tags: # Resource tags
      environment: production
      team: platform
  deletionPolicy: retain # Deletion behavior (retain or delete)
  lifecycle: # Lifecycle rules
    preventDeletion: true # Prevent accidental deletion
    requireApproval: true # Require manual approval
    driftDetection: true # Enable drift detection
status: # Status
  phase: Ready # Current status
  resourceId: vpc-0a1b2c3d4e5f # Provisioned resource ID
  message: "VPC is in sync" # Status message
```

RECONCILIATION FLOW

- 1 Read CloudInfrastructure CR and validate specification
- 2 Check existing cloud resource (external state)
- 3 Compare desired state vs actual state
- 4 Create, update or delete resource (idempotent)
- 5 Apply tags, configuration and lifecycle rules
- 6 Handle deletion safeguards and approval (if enabled)
- 7 Reconcile status, events and metrics
- 8 Detect and correct drift automatically

PRODUCTION CONSIDERATIONS

- ✓ Use least privilege cloud credentials
- ✓ Implement deletion safeguards
- ✓ Enable drift detection and reconciliation
- ✓ Handle API rate limits and retries
- ✓ Use tagging and ownership boundaries
- ✓ Provide audit logs and detailed events
- ✓ Support multi-cloud and multi-region
- ✓ Monitor costs, quotas and limits

FAILURE SCENARIOS

- ! Cloud API failure or rate limit exceeded
- ! Invalid configuration or unsupported region
- ! Accidental deletion without approval
- ! Drift detected and remediation fails
- ! Insufficient permissions (IAM/RBAC)
- ! Resource quota or limit reached
- ! Network connectivity to cloud provider lost
- ! Partial resource creation (inconsistent state)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

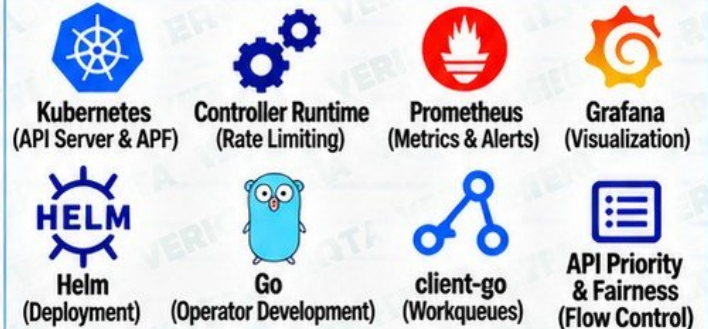
17 Build a Kubernetes API Server Load and Controller Backpressure Manager

Detect reconciliation storms, regulate controller concurrency, respect API Priority and Fairness, and reduce control-plane pressure.

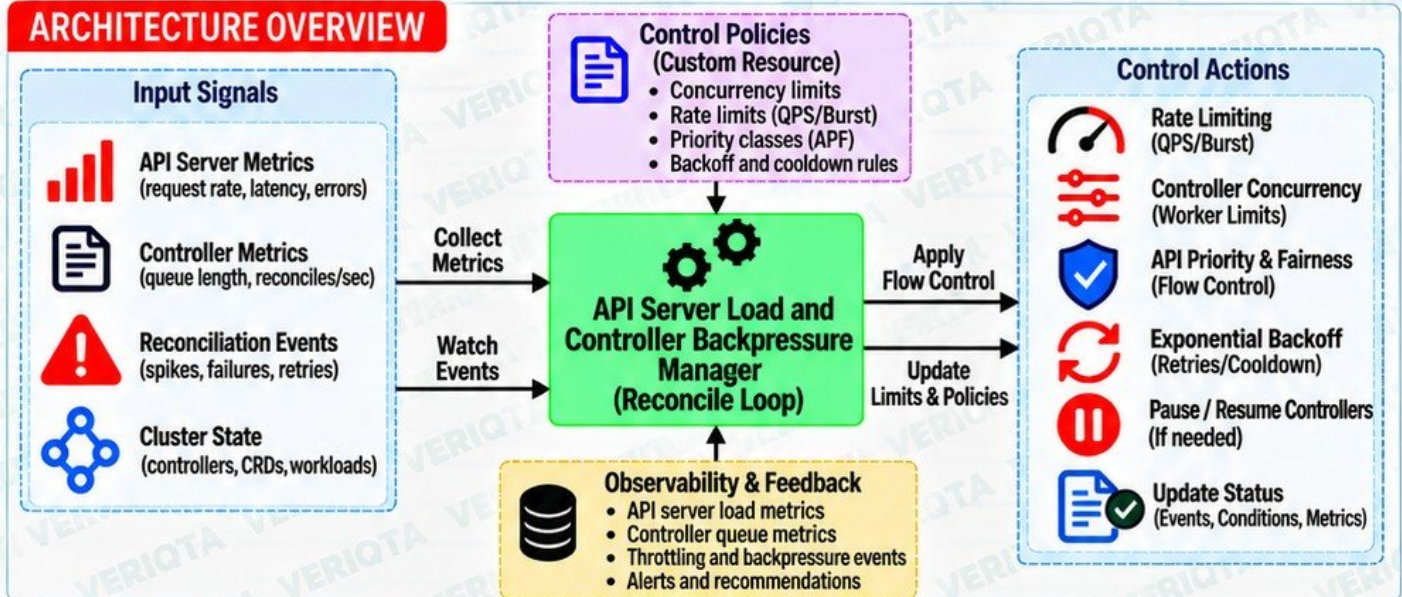
PROJECT OBJECTIVE

- ✓ Detect reconciliation storms and API overload
- ✓ Monitor API server load and controller behavior
- ✓ Regulate controller concurrency and request rates
- ✓ Respect Kubernetes API Priority and Fairness (APF)
- ✓ Reduce control-plane pressure and prevent outages
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1      # API version
kind: APILoadPolicy            # Resource type
metadata:                     # Metadata
  name: control-plane-protection # Resource name
  namespace: platform          # Namespace
spec:                         # Policy specification
  targets:                    # Target controllers
    - name: main-controllers
      concurrencyLimit: 5      # Max concurrent workers
      qps: 20                  # Requests per second
      burst: 40                # Burst limit
      priorityClass: "workload-low" # APF priority class
  backoff:                    # Backoff configuration
    initialDelay: 5s
    maxDelay: 5m              # Maximum backoff delay
  alerting:                   # Alert configuration
    enable: true
    errorRateThreshold: 0.05  # Error rate threshold (5%)
  actions:                    # Automated actions
    pauseOnCritical: true     # Pause controllers if critical
    autoScaleDown: true      # Scale down controllers
```

RECONCILIATION FLOW

- 1 Monitor API server metrics and controller queues
- 2 Detect reconciliation storms and abnormal load
- 3 Evaluate against configured thresholds and policies
- 4 Apply rate limiting and concurrency controls
- 5 Respect API Priority and Fairness settings
- 6 Pause or slow controllers when necessary
- 7 Update controller limits and backoff settings
- 8 Record status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use realistic thresholds and tune for your environment
- ✓ Respect Kubernetes API Priority and Fairness
- ✓ Prevent controller starvation and ensure fairness
- ✓ Provide detailed metrics, events and audit logs
- ✓ Test under simulated load and reconciliation storms
- ✓ Ensure critical controllers are protected
- ✓ Implement automatic recovery and rollback
- ✓ Monitor long-term control-plane health

FAILURE SCENARIOS

- ❗ Reconciliation storm from misconfigured controller
- ❗ API server high latency or timeouts
- ❗ Controller queue growing uncontrollably
- ❗ APF misconfiguration causing request starvation
- ❗ Rate limits too high or too low
- ❗ Controllers stuck due to excessive retries
- ❗ Control-plane resource exhaustion (CPU/memory)
- ❗ Loss of metrics or alerting
- ❗ Incorrect policy causing important controllers to pause



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18 Build a Distributed Leader Election and Fencing Controller

Coordinate exclusive workload ownership using Kubernetes Leases, fencing tokens, lease-expiry handling, and failure recovery.

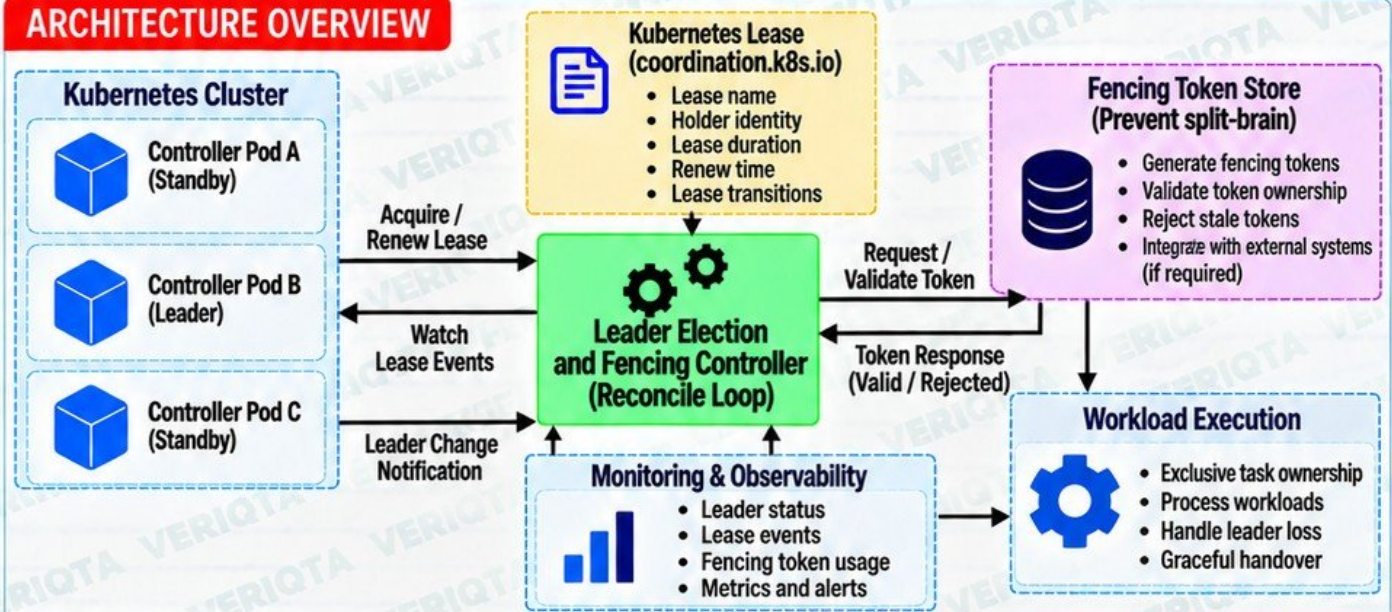
PROJECT OBJECTIVE

- ✓ Implement leader election using Kubernetes Leases
- ✓ Coordinate exclusive workload ownership
- ✓ Use fencing tokens to prevent split-brain operations
- ✓ Handle lease-expiry and leader transitions
- ✓ Recover gracefully from failures and network issues
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```

apiVersion: veriqta.io/v1
kind: LeaderElection
metadata:
  name: workload-controller
  namespace: production
spec:
  leaseName: workload-lease
  leaseDurationSeconds: 15
  renewDeadlineSeconds: 10
  retryPeriodSeconds: 2
  fencing:
    enabled: true
    tokenStore:
      type: redis
      address: redis:6379
  workloads:
    - name: data-processor
      concurrency: 1
      failoverStrategy: graceful
  metrics:
    enabled: true
  events:
    enabled: true
  
```

API version
 # Resource type
 # Metadata
 # Resource name
 # Target namespace
 # Specification
 # Lease name
 # Lease duration (seconds)
 # Renew deadline (seconds)
 # Retry period (seconds)
 # Fencing configuration
 # Enable fencing tokens
 # External token store
 # Store type (e.g. redis)
 # Store address
 # Managed workloads
 # Workload name
 # Single replica (leader only)
 # Failover strategy
 # Metrics configuration
 # Enable metrics
 # Event configuration
 # Enable events

RECONCILIATION FLOW

- 1 Start controller and try to acquire Kubernetes Lease
- 2 Validate or generate fencing token
- 3 Become leader and start exclusive workload processing
- 4 Renew lease and update fencing token periodically
- 5 Watch for lease changes and handle leader loss
- 6 Prevent split-brain using fencing token validation
- 7 Transfer ownership on failure or lease expiry
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Tune lease duration and renew deadlines
- ✓ Use external token store for fencing
- ✓ Handle clock skew and network partitions
- ✓ Implement graceful shutdown on leader loss
- ✓ Monitor lease transitions and token usage
- ✓ Set up alerts for frequent leader changes
- ✓ Test failure scenarios and recovery
- ✓ Ensure idempotent workload processing

FAILURE SCENARIOS

- ! Leader pod crash or termination
- ! Network partition between controllers
- ! Lease not renewed (expiry)
- ! Stale fencing token usage
- ! Split-brain (multiple leaders)
- ! Token store unavailable
- ! API server latency or throttling
- ! Slow handover causing workload delay



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

18 Build a Distributed Leader Election and Fencing Controller

Coordinate exclusive workload ownership using Kubernetes Leases, fencing tokens, lease-expiry handling, and failure recovery.

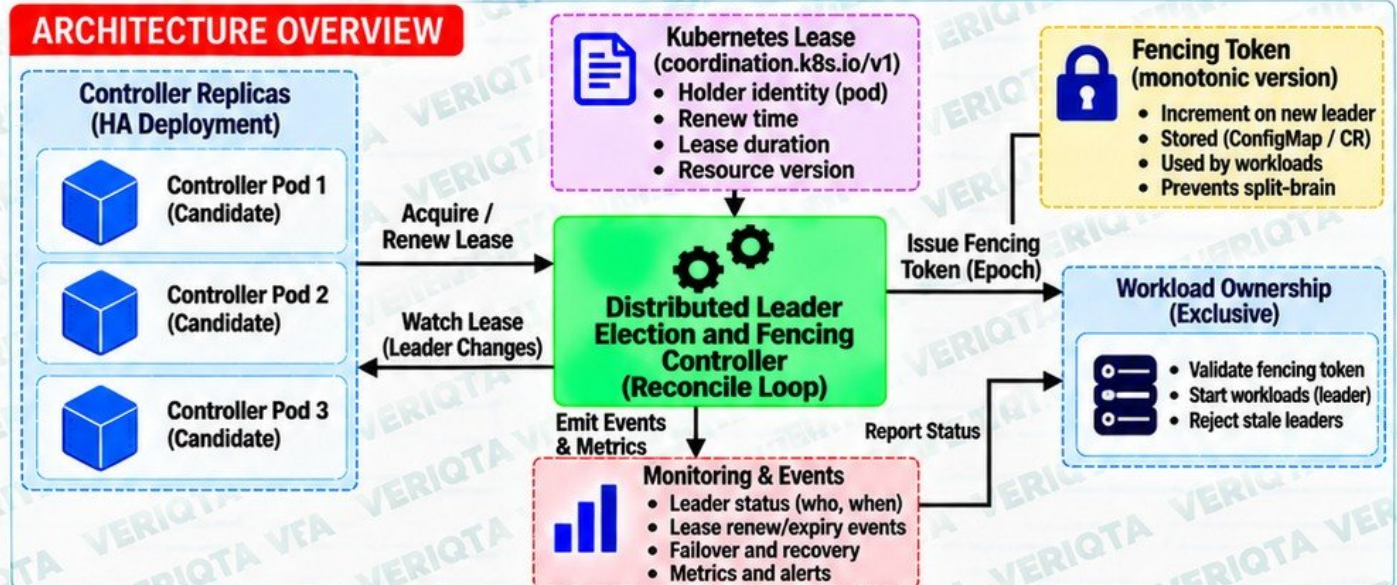
PROJECT OBJECTIVE

- ✓ Implement leader election using Kubernetes Leases
- ✓ Enforce exclusive workload ownership
- ✓ Use fencing tokens to prevent split-brain
- ✓ Handle lease expiry and leader failover
- ✓ Recover safely from failures and network issues
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```

apiVersion: veriqta.io/v1
kind: LeaderElectionPolicy
metadata:
  name: job-controller
  namespace: production
spec:
  lease:
    name: job-controller-lease
    namespace: production
    durationSeconds: 15
    renewDeadlineSeconds: 10
    retryPeriodSeconds: 2
  fencing:
    enabled: true
    storage: configmap
    tokenKey: epoch
  workload:
    type: JobProcessor
    startCommand: "/app/start"
  metrics:
    enabled: true
  events:
    enabled: true
  
```

RECONCILIATION FLOW

- 1 Read custom resource and lease configuration
- 2 Join leader election using Kubernetes Lease
- 3 Periodically renew lease if leader
- 4 Acquire new leader if lease expires
- 5 Generate and store fencing token (epoch)
- 6 Start or stop workloads based on leadership
- 7 Handle failures, retries and network partitions
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Use stable, unique leader identity (pod UID)
- ✓ Choose appropriate lease durations
- ✓ Persist fencing tokens externally (CM/DB)
- ✓ Validate fencing token in workloads
- ✓ Handle clock skew and network partitions
- ✓ Monitor leader changes and failovers
- ✓ Test failover and recovery scenarios

FAILURE SCENARIOS

- ! Leader pod crashes unexpectedly
- ! Network partition (split-brain risk)
- ! Lease renew failures (API unavailable)
- ! Clock skew causing early expiry
- ! Stale leader without fencing
- ! Token store unavailable
- ! Multiple leaders (misconfiguration)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

19 Build a Self-Service Internal Developer Platform Operator

Provision application environments through platform APIs, enforce golden paths, manage dependencies, and implement lifecycle governance.

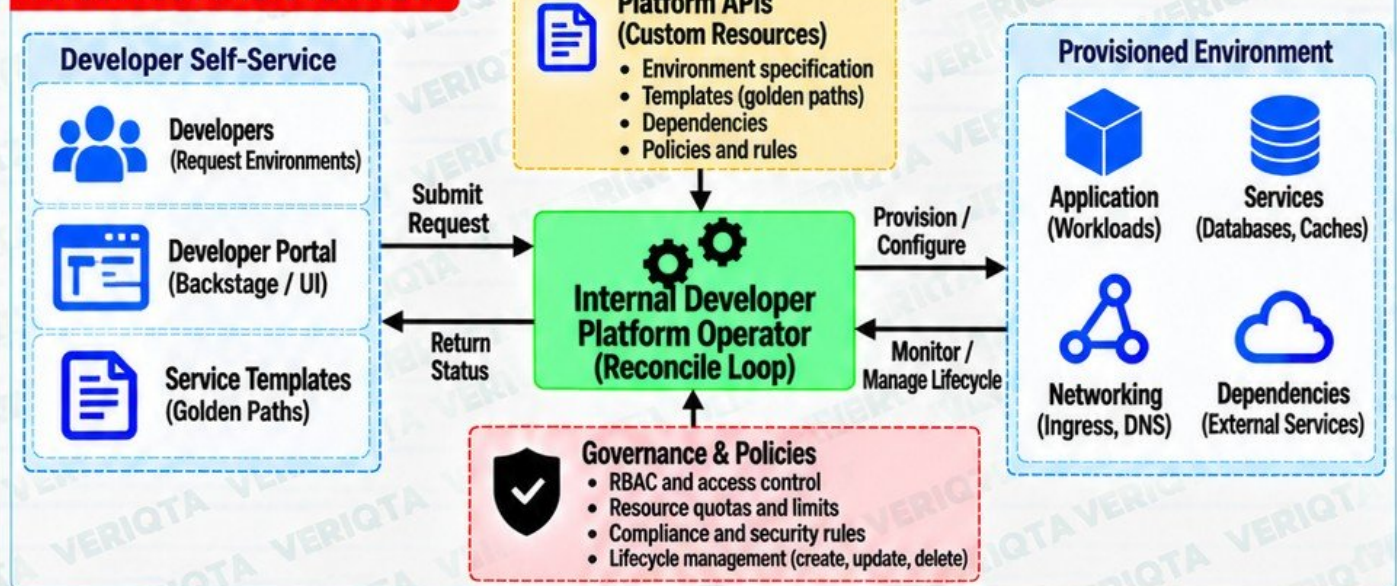
PROJECT OBJECTIVE

- ✓ Provision application environments via platform APIs
- ✓ Enforce golden paths and standards
- ✓ Manage dependencies and integrations
- ✓ Implement lifecycle governance
- ✓ Provide self-service for developers
- ✓ Ensure security, compliance and cost control
- ✓ Provide clear status, events and metrics

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: platform.veriqta.io/v1 # API version
kind: DeveloperEnvironment          # Resource type
metadata:                           # Metadata
  name: web-app-dev                 # Resource name
  namespace: development            # Target namespace
spec:                               # Owner team
  owner: developer-team             # Environment template
  template: web-app-golden-path     # Configuration parameters
  parameters:                       # Number of replicas
    replicas: 2                     # Include database
    database: true                  # Include cache
    cache: redis                    # Expose via ingress
    ingress: true                   # Resource requests/limits
  resources:                         # CPU request
    cpu: "500m"                    # Memory request
    memory: "1Gi"                  # Required dependencies
  -dependencies:                    # Database dependency
    - name: postgres               # Database version
      version: "15"                # Cache dependency
    - name: redis                   # Cache version
      version: "7"                  # Lifecycle settings
  lifecycle:                         # Environment TTL (days)
    ttlDays: 30                    # Auto delete after TTL
    autoDelete: true                # Labels/tags
  tags:                             # Team tag
    team: platform                 # Environment tag
    environment: dev
```

RECONCILIATION FLOW

- 1 Developer submits environment request via portal/API
- 2 Validate request against golden path and policies
- 3 Resolve and check dependencies
- 4 Provision Kubernetes resources and external services
- 5 Configure networking, security and access
- 6 Monitor environment health and usage
- 7 Enforce lifecycle rules (update, suspend, delete)
- 8 Provide status, events and metrics to developer

PRODUCTION CONSIDERATIONS

- ✓ Use standardized and approved templates
- ✓ Enforce RBAC, quotas and resource limits
- ✓ Validate dependencies before provisioning
- ✓ Implement cost controls and lifecycle policies
- ✓ Monitor environment usage and health
- ✓ Provide audit logs and detailed events
- ✓ Ensure secure and compliance standards
- ✓ Support cleanup and automatic deletion
- ✓ Test failure scenarios and recovery processes
- ✓ Document golden paths and usage guidelines

FAILURE SCENARIOS

- ❗ Invalid or non-compliant environment request
- ❗ Dependency service unavailable
- ❗ Resource quota exceeded
- ❗ Provisioning failure (network, permissions)
- ❗ Misconfigured template or parameters
- ❗ Security policy violation (RBAC)
- ❗ Environment deletion failure
- ❗ Orphaned resources after TTL
- ❗ External service integration failure
- ❗ Portal or API outage



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

20 Build an Autonomous Kubernetes Incident Remediation Operator

Correlate incidents, evaluate remediation policies, execute bounded recovery actions, enforce approval gates, and verify service restoration.

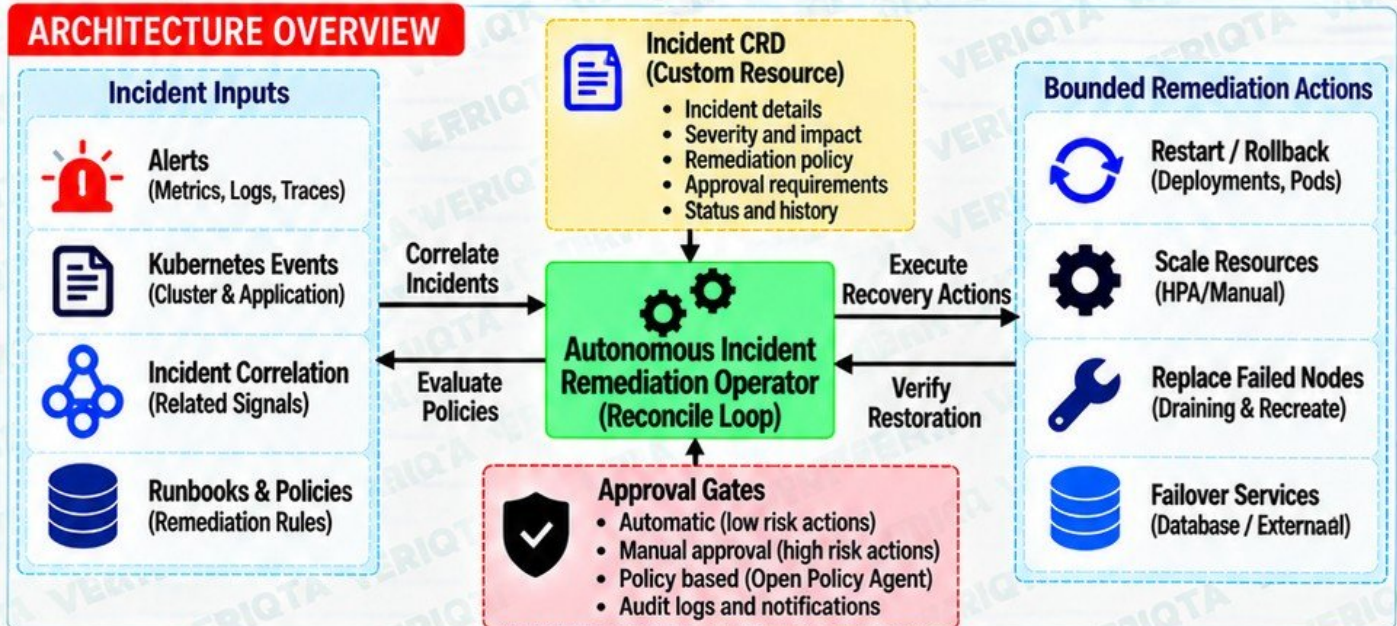
PROJECT OBJECTIVE

- ✓ Correlate alerts, events and incidents
- ✓ Evaluate remediation policies and runbooks
- ✓ Execute bounded and safe recovery actions
- ✓ Enforce approval gates (manual or automatic)
- ✓ Verify service restoration and stability
- ✓ Prevent repeated incidents (post-incident actions)
- ✓ Provide clear status, events and audit logs

KEY TECHNOLOGIES



ARCHITECTURE OVERVIEW



CUSTOM RESOURCE EXAMPLE

```
apiVersion: veriqta.io/v1 # API version
kind: IncidentRemediation # Resource type
metadata: # Metadata
  name: api-outage # Resource name
  namespace: production # Namespace
spec: # Specification
  incidentRef: # Related incident
    name: api-high-error-rate
  severity: high # Severity (low|medium|high)
  remediationPolicy: # Policy reference
    name: restart-and-scale
  approvalGate: # Approval requirement
    type: manual
    approvers: ["sre-team"]
  actions: # Remediation actions
    - type: restart # restart | rollback | scale
      target: deployment/api # restart & failure
    - type: scale # resource scaling
      target: hpa/api # auto scale
      minReplicas: 3 # auto collapse
      maxReplicas: 10
  verification: # Verification checks
    - type: http # http | metric | custom
      endpoint: /health
      rollbackOnFailure: true # Rollback if remediation fails
```

RECONCILIATION FLOW

- 1 Detect and correlate incident signals
- 2 Create or update Incident CR
- 3 Evaluate remediation policy and risk level
- 4 Check approval gate (automatic or manual)
- 5 Execute bounded recovery actions
- 6 Monitor service health and verify restoration
- 7 Rollback if verification fails
- 8 Update status, events and metrics

PRODUCTION CONSIDERATIONS

- ✓ Define safe and bounded remediation actions
- ✓ Implement approval policies and RBAC
- ✓ Test remediation runbooks in non-production
- ✓ Verify recovery actions and health checks
- ✓ Maintain audit logs and notifications
- ✓ Prevent repeated incidents (post-remediation)
- ✓ Integrate with on-call and communication tools
- ✓ Set limits, timeouts and rollback mechanisms

FAILURE SCENARIOS

- ❗ Incorrect remediation action executed
- ❗ Approval gate bypass or misconfiguration
- ❗ Remediation action causes further outage
- ❗ Verification check fails or is incorrect
- ❗ Rollback fails after remediation
- ❗ Policy misconfiguration (wrong target)
- ❗ Missing permission (RBAC)
- ❗ Repeated remediation loops
- ❗ External dependency not recovered
- ❗ Notification failure (no alert to on-call)



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta